Final Arquitectura de computadoras

Unidad 1 : Arquitectura y Organización de Computadoras

Concepto de Arquitectura. Relación con Organización de Computadoras. Repaso del modelo de Von Neumann. Descripción del funcionamiento de un sistema basado en un microprocesador. Buses, teoría de operación, buses sincrónicos y asincrónicos. Ejemplos. Repaso de ejecución de instrucciones. Ejecución solapada ("pipeline"). Su aplicación en procesadores contemporáneos. Análisis de prestaciones. Arquitecturas reconfigurables: conceptos. Sistemas embebidos: conceptos.

Unidad 2 : Subsistema Unidad Central de Procesos

Repaso de máquinas que ejecutan instrucciones. Ejemplificación en procesadores típicos: IA32. Análisis del conjunto de instrucciones de procesadores de uso comercial. Concepto de máquinas CISC y RISC. Lineamientos básicos en el diseño de un procesador RISC. Análisis de prestaciones. Ejemplos: procesadores MIPS y ARM. Interrupciones: tratamiento general. Interrupciones por software y por hardware, vectores, descripción y tratamiento particular de cada una. Relación entre las interrupciones y el manejo de operaciones de E/S.

Unidad 3 : Subsistema E/S

Concepto de E/S y su relación con la CPU, tipos de puertas. Concepto de puerta de Entrada y Salida paralelo. Concepto de puerta de Entrada y Salida serie. Tipos de transmisión serie. Descripción del formato de transmisión serie asincrónica y sincrónica. Descripción funcional de una puerta de E/S serie asincrónica, acceso a registros internos para control y determinación del estado de operación de la puerta. Mapeado del subsistema E/S y la memoria. Administración de las puertas por encuesta (polling) o por interrupción. Tratamiento de la CPU de las operaciones de E/S, por interrupción o por software. Transferencias de E/S por hardware, DMA, implementación.

Unidad 4 : Subsistema Memoria

Repaso de la organización jerárquica de la memoria, memoria principal y memoria secundaria. Memoria caché, concepto y descripción, análisis de prestaciones, métodos de implementación típicos, múltiples niveles. Ejemplos. Conceptos de memoria virtual.

Unidad 5 : Paralelismo y mejora de prestaciones

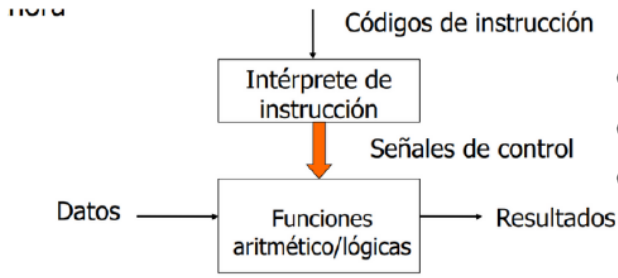
Concepto de procesamiento paralelo. Paralelismo a nivel instrucción. Procesadores superescalares. Ejemplos. Clasificación de arquitecturas paralelo: taxonomía de Flynn. Ejemplos de aplicación. Arquitecturas Multiprocesador. Memoria compartida o distribuida. Análisis de prestaciones..

Teoría PowerPoint

Clase 01-Repaso de oc

Programación en hardware: cuando cambiamos las tareas, debemos cambiar el hardware
Programación en software: en cada paso se efectúa alguna operación sobre los datos

CONCEPTO DE PROGRAMA



- Para cada paso se necesita un nuevo conjunto de señales de control.
- Las instrucciones proporcionan esas señales de control
- No hay que cambiar el hadware

ARQUITETURA VON NEUMANN

- La unidad central de procesamiento (CPU) está constituida por la unidad de control (UC) y la unidad aritmético-lógica (ALU).
- Datos e instrucciones deben introducirse en el sistema y los resultados se proporcionarán mediante componentes de entrada/salida (E/S).
- Se necesita almacenar temporalmente datos e instrucciones: Memoria Principal

REPERTORIO DE INSTRUCCIONES

- Es el conjunto completo de instrucciones que se realizan en una CPU.
  - Código máquina • Binario
- Representado simbólicamente por un conjunto de códigos de ensamblaje
  - de operaciones:
    - ADD (sumar), SUB (restar), LOAD (cargar datos en un registro)
  - de operandos:
    - ADD BX, PEPE; sumar contenidos de reg BX y dirección PEPE, el resultado se guarda en reg BX

ELEMENTOS DE UNA INSTRUCCION

- Código de operación ("Cod Op")
- Referencia a operandos fuentes
- Referencia al operando resultado
- Referencia a la siguiente instrucción

$$A = \frac{(B + C) \times D}{E}$$

Máquina de 0 direcciones

COD OP

Máquina de 1 dirección

COD OP	dirOP2

Máquina de 2 direcciones

COD OP	dirOP1/dirRES	dirOP2

Máquina de 3 direcciones

COD OP	dirOP1	dirOP2	dirRES

Máquina de 4 direcciones

COD OP	dirOP1	dirOP2	dirRES	dirProxInstr

DONDE SE ALMACENAN OPERANDOS?

- Memoria principal (o memoria virtual o en memoria cache)
- Registro de la CPU
- Dispositivo de E/S
- Almacenamiento tipo pila
- Almacenamiento tipo acumulador
- Almacenamiento tipo memoria-memoria
- Almacenamiento tipo registro-registro

TIPOS DE INSTRUCCIONES

- Procesamiento de datos:
  - instrucciones aritmético-lógicas
- Almacenamiento de datos:
  - instrucciones de memoria
- Transferencia de datos:
  - instrucciones de E/S
- Control:
  - instrucciones de testeo y flujo del programa

Registros vs Direcciones : Las operaciones entre los registros son más rápidas.

ORDEN DE LOS BYTES

Big endian: el byte más significativo en la dirección con valor numérico más bajo

Little endian: el byte menos significativo en la dirección con valor numérico más bajo

	BIG ENDIAN	LITTLE ENDIAN
Dir. de byte	Forma 1	Forma 2
00	98	32
01	76	54
02	54	76
03	32	98

Transferencia de datos

Debe especificarse:

- Ubicación del operando fuente
- Ubicación del operando destino
- Tamaño de los datos a ser transferidos
- Modo de direccionamiento

MODOS DE DIRECCIONAMIENTO

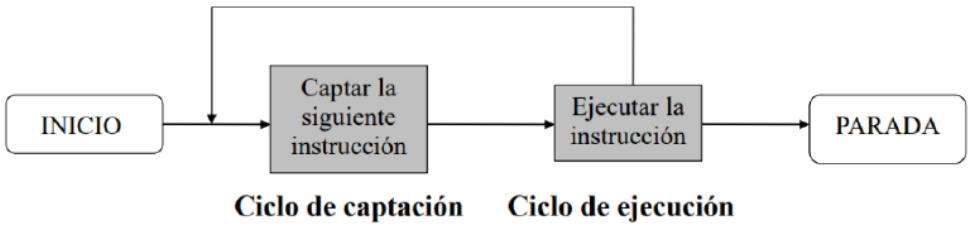
- Inmediato
- Directo de memoria o Absoluto
- Directo de Registro
- Indirecto de memoria (en desuso)
- Indirecto con registro
- Indirecto con Desplazamiento : basado, indexado o relativo al PC , Pila (o relativo al SP)

Subrutinas

- Innovación en lenguajes de programación
- Programa auto-contenido
- Puede invocarse desde cualquier punto de un programa
  - mediante instrucción CALL
- Brinda economía (código usado varias veces) y modularidad (subdivisión en unidades pequeñas).
- Requiere pasaje de argumentos (parámetros)
  - por valor (copia de una variable)
  - por referencia (dirección de la variable)

Ciclo de instrucción

Dos pasos: ciclo de captación y ciclo de ejecución



CICLO DE CAPTACIÓN

- La dirección de la instrucción que se debe captar se encuentra en el registro Contador de Programa (PC)
- La UC **capta** la instrucción desde la Memoria
  - La instrucción va al registro de instrucción (IR)
- El registro PC se incrementa
  - a no ser que se indique lo contrario.
- La UC interpreta la instrucción captada y debe lleva a cabo la acción requerida

CICLO DE EJECUCIÓN

Acciones posibles:

- Procesador - memoria
  - Transferencia de datos CPU - Memoria.
- Procesador - E/S
  - Transferencias de datos CPU y módulo de E/S.
- Procesamiento de datos
  - Alguna operación aritmética o lógica con los datos.
- Control
  - Alteración de la secuencia de ejecución.
    - Instrucción de salto

ó combinación de las acciones anteriores

Clase 02-entrada/salida

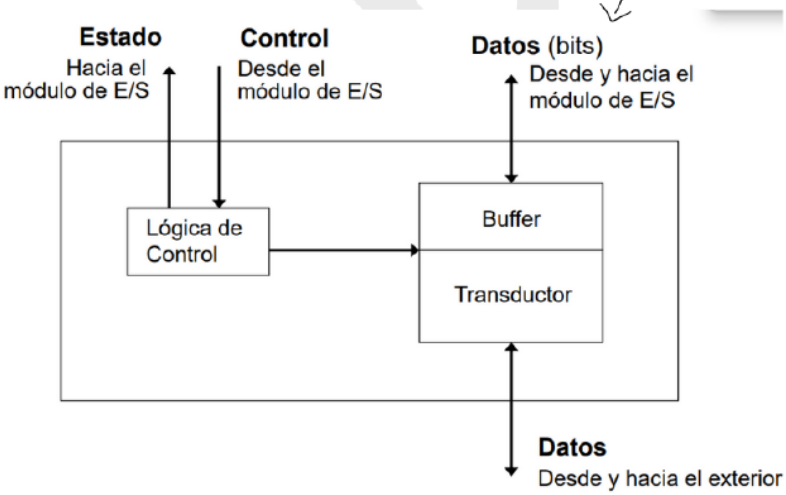
PROBLEMAS DE E/S

Variedad de periféricos: Existen diferentes dispositivos externos que operan con diversos métodos, formatos y velocidades. Los periféricos son más lentos que la CPU y la RAM, lo que requiere módulos de E/S con cierta "inteligencia" para manejar esta diferencia.

MODULO DE E/S

Realiza la interfaz entre el procesador y la memoria (bus) y los periféricos. Pueden manejar uno o más periféricos.

DISPOSITIVO EXTERNO



OPERACION DE ENTRADA O SALIDA

Requiere:

- Direccionamiento: E/S puede estar mapeada en memoria (compartiendo espacio de direcciones con la memoria) o aislada (con espacios de direcciones separados y órdenes específicas).
- Transferencia de datos: Lectura o escritura.
- Gestión: Sincronización y control de las transferencias.

Funciones de un modulo de e/s

Don't Forget

permite la interacción entre la CPU, la memoria y los periféricos. Estas son las principales funciones que realiza:

1. Control y temporización de dispositivos externos:
   - Coordina y sincroniza las operaciones de entrada/salida de uno o más periféricos.
2. Interpretación de órdenes:
   - Recibe las instrucciones de la CPU y las traduce en comandos que los periféricos pueden entender.
3. Comunicación con la CPU y la memoria:
   - Interactúa con los registros de la CPU para recibir instrucciones y con la memoria para realizar transferencias de datos.
4. Control de transferencias de datos:
   - Convierte formatos y adapta velocidades entre la CPU y los periféricos.
5. Comunicación con dispositivos periféricos:
   - Envía comandos y recibe datos de los periféricos conectados.
6. Gestión del estado del periférico:
   - Monitorea y reporta a la CPU el estado de los dispositivos (por ejemplo, si están listos o no para operar).
7. Buffering de datos:
   - Realiza almacenamiento temporal de datos para minimizar las diferencias de velocidad entre los periféricos y el sistema.
8. Detección de errores:
   - Verifica si hubo problemas durante las operaciones de entrada/salida y notifica a la CPU.

CAPACIDADES DE UN MODULO DE E/S

- ocultar las propiedades del dispositivo a la CPU
- ocuparse de uno o varios dispositivos
- controlar o no las funciones del dispositivo

DIRRECIONAMIENTO DE E/S

- E/S asignada en memoria: Dispositivos de E/S y memoria comparten un único espacio de direcciones con la memoria.
- E/S aislada: Espacios de direcciones separados, con instrucciones específicas.

EVOLUCIÓN DE LOS CANALES DE E/S

Los dispositivos de E/S se han vuelto más sofisticados, llevando a una evolución en cómo se manejan:

1. CPU controla directamente los periféricos.
2. Se introduce un módulo de E/S o controlador para gestionar las operaciones.
3. Se agregan interrupciones al módulo de E/S.
4. El módulo de E/S incorpora DMA para transferencias más rápidas.
5. El módulo de E/S incluye su propio procesador y un pequeño conjunto de instrucciones.
6. Finalmente, el módulo de E/S incorpora su memoria local, convirtiéndose prácticamente en una computadora independiente.

Características de Canales de E/S

- Representan una extensión avanzada del DMA.
- Pueden ejecutar instrucciones de E/S completas.
- Asumen el control completo de la transferencia de datos, descargando completamente a la CPU.
- La CPU solo inicia la transferencia y ordena la ejecución del programa almacenado en memoria que define:

1. Dispositivos a usar.
2. Áreas de memoria.
3. Prioridades y manejo de errores.

Técnicas de gestión de e/s

E/S Programada con espera de respuesta

En este método, la CPU controla directamente la operación de entrada/salida. Los pasos clave son:

- Proceso:
  a. La CPU solicita una operación al módulo de E/S (por ejemplo, leer o escribir datos).
  b. El módulo de E/S realiza la operación y actualiza su estado.
  c. La CPU verifica constantemente (en un bucle) si el dispositivo está listo (por ejemplo, si terminó de escribir o tiene datos para leer).
  d. Una vez que el dispositivo completa la operación, la CPU procesa los datos.
- Ventajas:
  - Sencillo de implementar.
  - No requiere hardware adicional.
- Desventajas:
  - La CPU permanece ociosa mientras espera que el dispositivo termine, lo que resulta ineficiente.
  - Este método es adecuado solo para operaciones de E/S muy simples o dispositivos rápidos.

E/S con interrupciones

Este método mejora la eficiencia permitiendo que la CPU continúe trabajando mientras el módulo de E/S realiza la operación. Cuando el módulo de E/S está listo, envía una interrupción a la CPU.

- Proceso:
  a. La CPU envía una orden al módulo de E/S para realizar una operación (por ejemplo, leer o escribir).
  b. El módulo de E/S realiza la operación en paralelo, mientras la CPU sigue ejecutando otras tareas.
  c. Cuando el módulo termina, envía una señal de interrupción a la CPU.
  d. La CPU guarda su contexto actual, gestiona la interrupción (por ejemplo, procesa los datos recibidos) y luego retoma su tarea previa.
- Ventajas:
  - Reduce significativamente el tiempo que la CPU pasa esperando.
  - Permite que la CPU realice otras tareas mientras espera.
- Desventajas:
  - Puede generar una sobrecarga de interrupciones si el dispositivo es muy rápido o hay múltiples periféricos.
  - La CPU debe realizar una gestión de interrupciones, lo que implica guardar y restaurar su contexto.

Acceso directo a memoria (DMA)

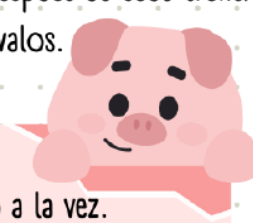
En esta técnica, se introduce un controlador de DMA que realiza la transferencia de datos entre el periférico y la memoria principal sin intervención de la CPU.

- Proceso:
  a. La CPU configura al controlador de DMA con los parámetros de la transferencia (dirección de memoria, tipo de operación, cantidad de datos, etc.).
  b. El controlador DMA se encarga de manejar directamente la transferencia entre el periférico y la memoria.
  c. Cuando la transferencia finaliza, el controlador envía una señal de interrupción a la CPU para informarle que terminó.
- Ventajas:
  - Libera a la CPU de la gestión directa de la transferencia.
  - Es más eficiente, especialmente para grandes volúmenes de datos.
  - Minimiza la carga de trabajo de la CPU.
- Desventajas:
  - Requiere hardware adicional (el controlador DMA).
  - Si el DMA usa intensivamente el bus del sistema, puede ralentizar el acceso de la CPU a la memoria.

MODOS DE TRANSFERENCIA DMA

Si el DMAC sólo toma el control del bus durante los intervalos de tiempo en los que la CPU no hace uso del mismo el rendimiento del sistema no sufrirá degradación alguna

1. Modo ráfaga (burst):
   - El DMA toma control del bus y no lo libera hasta completar la transferencia de todo el bloque de datos.
   - Ventaja: La transferencia es rápida.
   - Desventaja: Durante ese tiempo, la CPU no puede acceder al bus.
2. Modo robo de ciclo (cycle-stealing):
   - El DMA solicita el bus palabra por palabra, liberándolo después de cada transferencia.
   - Ventaja: La CPU puede seguir usando el bus en los intervalos.
   - Desventaja: La transferencia tarda más tiempo.



TIPOS DE CANALES DE E/S

1. SELECTOR:
   - Controla varios dispositivos de alta velocidad, pero solo uno a la vez.
   - Selecciona un dispositivo, realiza la transferencia y luego pasa al siguiente.
   - Ventaja: Alta velocidad para un dispositivo dedicado.
   - Desventaja: No puede manejar múltiples dispositivos simultáneamente.
2. MULTIPLEXOR:
   - Permite manejar varios dispositivos a la vez.
   - Tipos:
     - Multiplexor de bytes: Transfiere caracteres (datos pequeños).
     - Multiplexor de bloques: Intercala bloques de datos entre dispositivos.
   - Ventaja: Ideal para manejar periféricos de baja velocidad simultáneamente.
   - Desventaja: Más complejo que el selector.

CLASE 03-INTERRUPCIONES

INTERRUPCIONES:

- Mecanismo mediante el cual se puede interrumpir el procesamiento normal de la CPU. Ejecución secuencial de instrucciones de un programa
- Pueden ser de origen interno o externo a la CPU.

MOTIVOS PARA INTERRUMPIR:

- Por resultado de una ejecución de una instrucción
  - Ej: desbordamiento aritmético ("overflow"), división por cero
- Por un temporizador interno del procesador.
  - Permite al S.O. realizar ciertas funciones de manera regular.
- Por una operación de E/S.
  - Ej: para indicar la finalización normal de una operación.
- Por un fallo de hardware.
  - Ej: error de paridad en la memoria, pérdida de energía.

MANEJO DE INTERRUPCIONES:

Transferir el control a un gestor que:

1. Salve el estado del procesador.
2. Responda a la causa de la interrupción.
3. Restaure el estado original del procesador.
4. Decida si continuar o repetir la instrucción interrumpida.

INTERRUPCIONES POR HARDWARE:

- Generadas por dispositivos de E/S.
- Características:
- Son eventos externos, no planeados y asincrónicos.
- No están relacionadas con el proceso en ejecución.
- Conocidas como "Interrupt Request" (IRQ).

JERARQUIA DE INTERRUPCIONES:

Si hay múltiples fuentes que pueden solicitar interrupción se establece cuales son mas importantes.

Ser consideran

- No Enmascarables: las que NO pueden ignorarse
  - Indican eventos peligrosos o de alta prioridad.
- Enmascarables: pueden ser ignoradas.
  - Con instrucciones podemos inhibir la posible solicitud.

TRAPS/EXCEPCIONES:

Interrupciones por hardware generadas por el procesador ante eventos como:

- Condiciones excepcionales: Overflow en la ALU.
- Fallas de programa: Ejecución de instrucciones no definidas.
- Fallas de hardware: Ejemplo, error de paridad de memoria.
- Accesos no alineados ó a zonas de memoria protegidos

INTERRUPCIONES POR SOFTWARE:

Instrucciones explícitas que afectan al procesador como interrupciones hardware.

Usos:

- Llamadas a funciones del S.O.
- Permiten que subrutinas se carguen en cualquier lugar.

Ventajas:

- No requieren conocer la dirección de la rutina en tiempo de ejecución.

CICLO DE INTERRUPCION:

1. Verifica si hay una señal de interrupción.
2. Si no hay, se ejecuta la siguiente instrucción.
3. Si hay, guarda el contexto y ejecuta la rutina de interrupción.

INTERRUPCIONES MULTIPLES:

- Manejo secuencial: Inhabilita otras interrupciones hasta gestionar la actual.

Prioridades:

- Una interrupción de mayor prioridad puede interrumpir otra de menor prioridad.

RECONOCIMIENTO DE INTERRUPCIONES:

Métodos:

- Interrupciones Multinivel: Cada dispositivo tiene una entrada física, es costoso.
- Línea de interrupción única: Se debe "preguntar" a cada dispositivo si ha producido el pedido de interrupción (polling).
- Interrupciones Vectorizadas: El dispositivo coloca un identificador en el bus de datos.

CONTROLADOR DE INTERRUPCIONES (PIC):

- Gestiona múltiples solicitudes de interrupciones.
- Funcionamiento:

Usa registros internos como IMR, IRR, ISR para manejar prioridades y peticiones.

TABLA DE VECTORES DE INTERRUPCIÓN:

Relaciona tipos de interrupción (0-255) con procedimientos de atención.

Ejemplo:

Tipo 0: Finalización de ejecución del programa.

Tipo 3: Punto de parada para depuración.

CLASE 04-MEMORIA

JERARQUIA DE MEMORIA:

Objetivo:

- Lograr la velocidad del nivel más rápido al costo del nivel más barato.

Características:

- Niveles inferiores: Más grandes, lentos y baratos que los superiores.
- Necesidad de correspondencia de direcciones entre niveles

Propiedades:

- Inclusión: Los datos en un nivel deben estar también en niveles inferiores.
- Coherencia: Las copias de la misma información en diferentes niveles deben ser iguales.

Principio de localidad de referencias:

- Temporal: Datos usados recientemente serán reutilizados pronto.
- Espacial: Datos cercanos a los recientemente utilizados serán referenciados.

MEMORIA CACHÉ:

- Pequeña memoria rápida entre la CPU y la memoria principal.
- Funcionamiento:

1. La CPU solicita datos a la caché.
2. Si están, se obtienen rápido ("acierto").
3. Si no, se copia el bloque desde la memoria principal ("fallo").

CONCEPTOS BASICOS:

Acierto *(hit):* se encuentra en la caché el dato solicitado

Fallo *(miss):* **no se** encuentra en la caché el dato solicitado

- un bloque que contiene la palabra accedida se copia de la memoria principal a una línea de caché.
- ***Tiempo para servir un fallo:*** depende de la latencia y ancho de banda de la memoria principal.
  - ***Latencia:*** tiempo necesario para completar un acceso a memoria.
  - ***Ancho de banda:*** cantidad de información por unidad de tiempo que puede transferirse desde/hacia la memoria.
- Los **fallos de caché** se gestionan mediante hardware y causan que el procesador se detenga hasta que el dato esté disponible.

PRESTACIONES DE LA JERARQUÍA

Tiempo de acceso medio a memoria

$$T_{acceso} = T_{acierto} + T_{fallos_memoria}$$

$$T_{fallos_memoria} = \text{Tasa de fallos} \times \text{Penalización_fallo}$$

$$T_{acceso} = T_{acierto} + TF \times PF$$

Para mejorar las prestaciones

- Reducir el tiempo en caso de acierto ($T_{acierto}$)
- Reducir la tasa de fallos (TF)
- Reducir la penalización por fallo (PF)

POLÍTICA DE REEMPLAZOS

Algoritmos:

- LRU: Menos recientemente usado.
- FIFO: Primero en entrar, primero en salir.
- LFU: Menos frecuentemente usado.
- Aleatoria: Sustituye una línea al azar.

UBICACIÓN DE BLOQUES EN CACHÉ

Tipos de correspondencia:

- Directa: Cada bloque tiene una posición específica.
  Nº línea caché = Nº bloque ref. ***mod*** Nº líneas caché
- Totalmente asociativa: Un bloque puede almacenarse en cualquier lugar de la caché
- Asociativa por conjuntos: Un bloque puede almacenarse en un conjunto restringido de lugares en la caché

Un conjunto es un grupo de lineas de la caché.
Nº conjunto = Nº bloque ref. ***mod*** Nº conjuntos caché

POLÍTICA DE ESCRITURA: EN ACIERTO

Write-through (*Escritura inmediata*).

- Se actualizan simultáneamente la posición de la caché y de la memoria principal.
  - con múltiples CPU, observar el tráfico a memoria principal para mantener actualizada cada cache local.
  - se genera mucho tráfico y retrasa la escritura.

Write-back (*Post-escritura*).

- La información sólo se actualiza en la caché.
  - Se marca como actualizada ⇒ *bit de "sucio".*
  - La memoria principal se actualiza en el reemplazo y puede contener información errónea en algún momento

POLÍTICA DE ESCRITURA: EN FALLO

Write allocate

- La información se lleva de la memoria principal a la caché. Se sobreescribe en la caché
  - Habitual con write-back

No-write allocate

- El bloque no se lleva a la memoria caché. Se escribe directamente en la memoria principal.
  - Habitual con write-through

Clase 05- segmentación de instrucciones

Segmentación de cauce

- Definición: La segmentación de cauce (pipelining) organiza el hardware de la CPU para realizar múltiples operaciones simultáneamente.
- Funcionamiento: Divide la ejecución de instrucciones en etapas, permitiendo su ejecución paralela.
- Propósito: Aprovechar el paralelismo en el flujo secuencial de instrucciones.

Características

- Mejora de prestaciones: Optimización del hardware.
- Invisibilidad al programador.
- Uniformización de etapas:
- Todas las etapas deben ajustarse al tiempo de la etapa más lenta.
- Dependencia del diseño: Influyen el repertorio de instrucciones y la organización del hardware.

Tareas a realizar por ciclo

Etapas del ciclo:

1. Búsqueda (Fetch): Captación de la instrucción y actualización del PC.
2. Decodificación (Decode): Interpretación de la instrucción, obtención de operandos.
3. Ejecución (Execute): Operaciones en la ALU.
4. Acceso a memoria (Memory Access): Lectura/escritura en memoria si es necesario.
5. Almacenamiento (Writeback): Escritura de resultados en registros.

Prestaciones del cauce segmentado

Teórica: El máximo rendimiento es completar una instrucción con cada ciclo de reloj.

Si K es el número de etapas del cauce ⇒

Vel. procesador segmentado = Vel. secuencial x K

El incremento potencial de la segmentación del cauce es proporcional al número de etapas del cauce.

Incrementa la productividad (throughput), pero no reduce el tiempo de ejecución de la instrucción

- Teórica:
  - Velocidad proporcional al número de etapas del cauce.
- Limitaciones:
  - No todas las instrucciones usan todas las etapas.
  - No todas las etapas pueden trabajar en paralelo.
  - Saltos de control no están optimizados.

ATASCOS DE UN CAUCE (STALL)

- Definición: Situaciones que bloquean la ejecución una instrucción en el ciclo correspondiente.
- Causas:

1. Estructurales: Conflictos por uso del hardware.
2. Dependencia de datos (RAW, WAW, WAR):
   - RAW: Una instrucción necesita un dato generado por otra.
3. Dependencia de control: Dependen de saltos o bifurcaciones.

TIPOS DE DEPENDENCIAS DE DATOS

- RAW (Lectura después de escritura): Verdadera dependencia.
- WAW (Escritura después de escritura): Depende del orden de escritura.
- WAR (Escritura después de lectura): Antidependencia.

RIESGOS ESTRUCTURALES

- Ocurren cuando dos instrucciones necesitan el mismo recurso.
- Solución: Aumentar recursos o evitar conflictos con programación.

RIESGOS DE CONTROL

Surgen cuando una instrucción altera el PC y afecta el flujo del programa.

RIESGOS POR DEPENDENCIA DE DATOS

Condición en la que los operandos fuente o destino de una instrucción no están disponibles en el momento en que se necesitan en una etapa determinada del cauce.

Clase 06-Soluciones a atascos

Atascos de un cauce (stall)

Definición: Situaciones que impiden que una instrucción se ejecute en el ciclo correspondiente.

Tipos de atascos:

- Estructurales: Conflictos por recursos.
- Dependencia de datos (RAW): Una instrucción necesita un dato generado por otra.
- Dependencia de control: Depende de cómo se ejecute otra instrucción.

Efecto: Disminución del rendimiento teórico, con CPI > 1.

Soluciones a riesgos de datos

- Riesgo RAW: Surge cuando una instrucción necesita el resultado de otra.
- Técnicas hardware:
- Adelantamiento (Forwarding): El resultado se pasa directamente de una etapa a otra sin esperar a que se escriba.
- Técnicas software:
- Introducir instrucciones NOP para evitar conflictos.
- Reordenar instrucciones para maximizar la separación entre dependencias.

Predicción de saltos

Técnicas estáticas:

- Asumir que nunca se salta o que siempre se salta.

Técnicas dinámicas:

- Conmutador saltar/no saltar: Basado en la historia previa de las instrucciones.
- Tabla de historia de saltos (BTB):
  - Guarda la dirección de bifurcación, destino e información de la instrucción.

SALTO RETARDADO:

- El compilador rellena los huecos de retardo con instrucciones útiles

FLUJOS MÚLTIPLES:

- Precaptar múltiples opciones de salto en diferentes cauces.
- Requiere elegir el cauce correcto, pero puede provocar retardos y aumentar la complejidad.

BUFFER DE BUCLES:

- Memoria rápida que almacena pequeños bucles o saltos frecuentes.
- Eficaz para mejorar el rendimiento en bucles pequeños.

Soluciones a riesgos estructurales

- Simple: Replicar, segmentar ó realizar turnos para el acceso a las unidades funcionales en conflicto.
- Replicación de recursos hardware (e.g., duplicar sumadores).
- Separar memorias de instrucciones y datos.
- Acceso por turnos al banco de registros:
- Escrituras en la primera mitad del ciclo.
- Lecturas en la segunda mitad.

Soluciones a riesgos de control

Existe una penalización por salto

Instrucciones de salto:

- Incondicionales: Determinar la dirección de destino lo antes posible.
- Condicionales: Dependen de una condición basada en el resultado de una instrucción previa.

Soluciones:

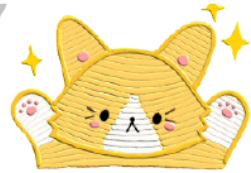
Hardware:

- Modificar la ruta de datos para resolver saltos en etapas tempranas (e.g., en la etapa de decodificación).
- Usar predicción de saltos.

Software:

- Salto retardado: El compilador introduce instrucciones útiles en los huecos de retardo (si no, usa NOP).

Otras soluciones hardware



Clase 07-Risc

CARACTERISTICAS:

- Registros de uso general y optimización por compiladores.
- Instrucciones limitadas y sencillas.
- Énfasis en optimizar la segmentación de instrucciones.

FINALIDAD E INCONVENIENTES DEL CISC:

- Ventajas: Simplificar la escritura de compiladores, soporte para lenguajes de alto nivel (HLL).
- Desventajas: Repertorio grande de instrucciones, costos más altos, y complejidad de hardware.

OPERACIONES:

- Asignaciones: Principalmente movimientos de datos.
- Condicionales y bucles: Controlan el flujo de ejecución.
- Llamadas a procedimientos: Requieren tiempo significativo debido al manejo de parámetros, variables locales y retornos.
- Traducción HLL a código máquina: Algunas instrucciones HLL resultan en muchas operaciones de máquina.

CARACTERÍSTICAS DE LA EJECUCIÓN:

Se analiza cómo los programas escritos en lenguajes de alto nivel (HLL) interactúan con el hardware:

1. Operaciones realizadas: Movimientos de datos, estructuras condicionales (IF, LOOP).
2. Procesador y memoria: Frecuencia de uso de operandos y modos de direccionamiento.
3. Secuenciamiento: Organización del control y del cauce de instrucciones.
4. Estudios dinámicos: Evaluación de estas características durante la ejecución.

LLAMADAS A PROCEDIMIENTOS:

- Consumen tiempo dependiendo del número de parámetros y el nivel de anidamiento.
- Localidad de referencias: La mayoría de las variables y operandos son locales.
- Ventana de profundidad: Se sugiere limitar la profundidad de las llamadas mediante un mejor manejo de registros y operandos.

VARIABLES GLOBALES:

Aunque el enfoque de ventanas de registros es eficiente para variables locales, presenta problemas para variables globales, que suelen requerir accesos frecuentes:

- Ineficiencia: Las variables globales se asignan a posiciones de memoria, lo que incrementa los tiempos de acceso.
- Solución propuesta:
  - Incorporar un conjunto de registros dedicados exclusivamente a variables globales.
  - Dividir los registros de la ventana actual en áreas específicas para manejar tanto locales como globales.

Banco de registros amplio	Cache
• Todos los datos escalares locales	• Datos escalares locales recientemente usados
• Variables individuales	• Bloques de memoria
• Variables globales asignadas por el compilador	• Variables locales y globales usadas recientemente
• Salvaguarda/restauración basadas en la profundidad de anidamiento	• Salvaguarda/restauración basadas en el algoritmo de reemplazo
• **Direccionamiento de registro**	• **Direccionamiento de memoria**

Amplio banco de registros
vs
cache

El análisis de programas y sus necesidades llevó a la conclusión de que un diseño eficiente de procesadores debe enfocarse en el uso de registros, especialmente para variables locales y temporales. Esto minimiza los accesos a memoria, que son costosos en términos de tiempo.

1. Optimización del acceso a variables locales:
   - La mayoría de las referencias a variables en los programas son locales y temporales, por lo que deben almacenarse en registros para reducir accesos a memoria.
   - Las llamadas a funciones presentan desafíos al manejar parámetros, devolver resultados y restaurar variables locales. Esto requiere un sistema eficiente.
2. Ventanas de registros:
   - Los registros del procesador se dividen en áreas específicas para:
     - Parámetros: Pasar datos entre funciones.
     - Datos locales: Variables propias de cada función.
     - Temporales: Cálculos intermedios.
   - Solapamiento: Los registros temporales de una función se superponen con los registros de parámetros de la siguiente función. Esto evita transferencias explícitas de datos entre llamadas, haciendo más eficiente el manejo de funciones anidadas.
3. Diseño del procesador con ventanas de registros:
   - Cada llamada a función activa un nuevo conjunto de registros.
   - El retorno a la función anterior restaura automáticamente los registros previos.
   - Este enfoque utiliza un buffer circular para administrar las ventanas, con punteros que indican qué conjunto de registros está en uso y cuál ha sido salvaguardado.
4. Beneficios del diseño RISC:
   - Velocidad: Al reducir accesos a memoria, las instrucciones se ejecutan más rápidamente.
   - Eficiencia: Se maximiza el uso de registros mediante un manejo estructurado.
   - Simplicidad: Un repertorio reducido de instrucciones simplifica el diseño del procesador, pero requiere que el compilador optimice la asignación de registros.

OPTIMIZACIÓN DE USO DE REGISTROS BASADA EN EL COMPILADOR

Dado que los procesadores solo tienen una cantidad limitada de registros (por ejemplo, 16 o 32), el compilador usa estrategias inteligentes para aprovecharlos al máximo:

- Registros virtuales: Al principio, el compilador asigna tantas variables como sea necesario a "registros simbólicos". Luego, mapea esos registros virtuales a los registros reales que tiene el procesador.
- Compartir registros reales: Si dos variables no se usan al mismo tiempo, pueden compartir un mismo registro real para ahorrar espacio.
- Coloreado de grafos: Es una técnica matemática que el compilador usa para decidir qué variable se asigna a qué registro, minimizando las veces que tiene que mover variables a memoria.

POR QUÉ SE CREÓ CISC Y SUS PROBLEMAS

CISC, o las computadoras de instrucciones complejas, surgieron con estas ideas:

1. Simplificar los compiladores: Con instrucciones más complejas, los compiladores podían traducir tareas complicadas del programador con menos pasos.
2. Programas más pequeños: Menos instrucciones reducían el tamaño de los programas, algo importante cuando la memoria era costosa.
3. Ejecución más rápida: La idea era que las instrucciones complejas hicieran más en menos tiempo.

Problema: En la práctica, esas instrucciones complejas hicieron que el hardware fuera más lento y complicado, especialmente para operaciones simples y frecuentes.

CARACTERÍSTICAS DEL RISC

- Instrucciones simples: Cada instrucción realiza solo una tarea y lo hace en un único ciclo de reloj, lo que lo hace más rápido.
- Registro a registro: Las operaciones trabajan directamente con registros, evitando accesos frecuentes a memoria.
- Diseño sencillo: Los procesadores tienen menos modos de direccionamiento y formatos más fáciles de manejar.

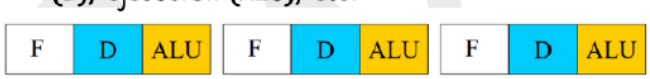
COMPARACIÓN ENTRE RISC Y CISC

- RISC prioriza la simplicidad y velocidad, ideal para ejecutar muchas instrucciones básicas rápidamente.
- CISC ofrece flexibilidad y soporte para tareas más complejas, pero a menudo es más lento y complicado.
- Hoy en día, los procesadores combinan ambos enfoques para equilibrar velocidad, flexibilidad y facilidad
- de uso.

Clase 08-Procesadores superescalares

PROCESADORES ESCALARES CARACTERÍSTICAS:

- Ejecución secuencial de instrucciones.
- Cada instrucción pasa por etapas: búsqueda (F), decodificación (D), ejecución (ALU), etc.



Instrucción 1 *Instrucción 2* *Instrucción 3*

Procesadores Superescalares: Permiten ejecutar múltiples instrucciones simultáneamente.

1. Duplicación de componentes como ALU y registros.
2. Permiten ejecutar varias instrucciones en paralelo:
3. Sumas, multiplicaciones, carga/almacenamiento, etc.

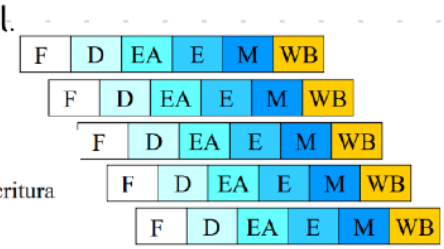
SEGMENTADO DE CAUSE:

Una unidad segmentada es una secuencia de etapas con la propiedad de que nuevas operaciones pueden iniciarse mientras otras están en proceso

Procesadores Supersegmentados: Dividen cada etapa del cauce en sub-etapas para mayor velocidad.

1. Subdivisión de las etapas del cauce.
2. Aumenta la frecuencia del reloj y el paralelismo.
3. No reduce el tiempo de ejecución individual, pero mejora el rendimiento general.

F: búsqueda
D: decodificación
EA: dirección efectiva
E: ejecución
M: memoria
WB: post escritura



PARALELISMO DE INSTRUCCIONES

- Definición: Las instrucciones independientes pueden ejecutarse en paralelo.
- Limitaciones:
  - Dependencias de datos.
  - Conflictos de recursos.
  - Dependencias de salida y antidependencias.

PARALELISMO DE LA MÁQUINA

- Factores que influyen:
  - Número de instrucciones captadas por ciclo.
  - Número de unidades funcionales disponibles.
  - Mecanismos para identificar instrucciones independientes:
    - Renombre de registros.
    - Ventanas de instrucciones.
- Políticas de emisión:
  - En orden: Decodificación y ejecución siguen el orden original.
  - Desordenada: Permite reordenar para aprovechar mejor los recursos.

POLÍTICAS DE EMISIÓN Y RIESGOS

- Ejemplo de un cauce superescalar:
  - Capta y decodifica 2 instrucciones a la vez.
  - Utiliza 3 unidades funcionales independientes.
- Riesgos:
  - RAW (Lectura después de escritura): Dependencia verdadera.
  - WAW (Escritura después de escritura): Dependencia de salida.
  - WAR (Escritura después de lectura): Antidependencia.

RENOMBRE DE REGISTROS

1. Problema:

- Las dependencias de salida (WAW) y las antidependencias (WAR) surgen porque los registros no pueden reflejar adecuadamente la secuencia de valores dictada por el flujo del programa.
- Esto genera conflictos de almacenamiento, que pueden detener alguna etapa del cauce.

2. Solución:

- Renombramiento de registros:
  - Los registros se asignan dinámicamente.
  - Las instrucciones posteriores se refieren a estos nuevos registros.
- De esta forma, los valores se almacenan en registros nuevos para evitar conflictos.

IMPLEMENTACIÓN SUPERESCALAR

- Requisitos:
  - Captación simultánea de múltiples instrucciones.
  - Lógica para identificar dependencias.
  - Emisión y ejecución paralela.
  - Mecanismos para garantizar el orden correcto del procesador.

EXCEPCIONES EN PROCESADORES SUPERESCALARES

- Excepciones imprecisas:
  - Cuando ocurre una excepción, pueden quedar instrucciones incompletas, generando estados inconsistentes.
- Solución:
  - Ejecución desordenada y finalización ordenada:
    - Instrucciones anteriores terminan correctamente.
    - Las que causaron la excepción y siguientes se abortan.
  - Uso de registros temporales (TW).

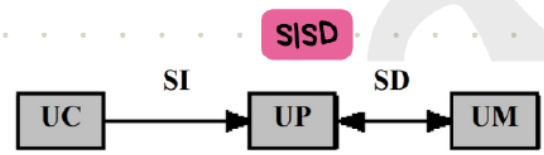
Clase 09- Procesamiento paralelo

INTRODUCCIÓN AL PROCESAMIENTO PARALELO

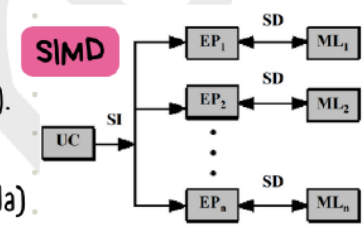
- Demanda constante de mayor rendimiento en máquinas.
- Dos enfoques para mejorar el rendimiento:
  a. Paralelismo a nivel instrucción (ILP) en un solo procesador.
  b. Paralelismo a nivel proceso mediante arquitecturas con varios procesadores.

CATEGORÍAS DE COMPUTADORAS

- Clasificación según Flynn:
  - SISD: Una secuencia de instrucciones y una secuencia de datos.
    a. Computadoras monoprocesador.
    b. Un único procesador ejecuta una secuencia de instrucciones sobre datos almacenados en una única memoria.

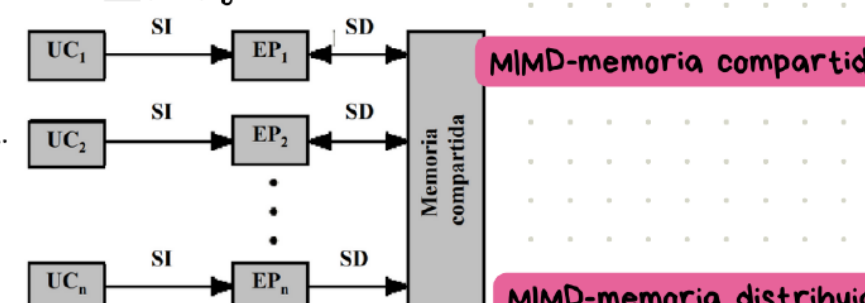


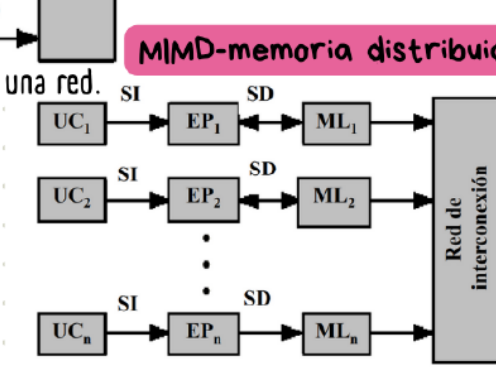
  - SIMD: Una secuencia de instrucciones y múltiples secuencias de datos.
    a. Una instrucción controla la ejecución simultánea en varios elementos de proceso (EP).
    b. Ejemplo: Procesadores vectoriales y matriciales.



  - MISD: Múltiples secuencias de instrucciones y una secuencia de datos (no implementada)
    a. Una secuencia de datos es procesada por múltiples procesadores, cada uno con una secuencia distinta de instrucciones.
    b. Nunca ha sido implementada.
  - MIMD: Múltiples secuencias de instrucciones y múltiples secuencias de datos.
    a. Varios procesadores ejecutan diferentes secuencias de instrucciones sobre datos distintos.
    b. Subdivisión:
      - Memoria compartida: Multiprocesadores simétricos (SMP) y sistemas NUMA.
      - Memoria distribuida: Clusters.

- Memoria compartida:
  - Procesadores comparten una memoria global.
  - Ventajas: Uniformidad en el acceso.
  - Ejemplo: SMP.
- Memoria distribuida:
  - Cada procesador tiene su propia memoria local y se comunican a través de una red.
  - Ejemplo: Clusters.

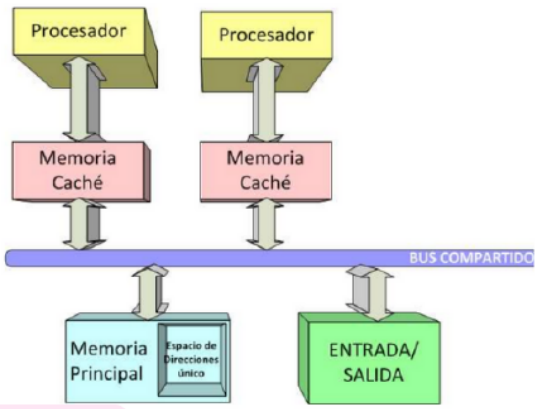




MULTIPROCESADORES SIMÉTRICOS (SMP)

- Características:
  - Dos o más procesadores similares comparten memoria principal y E/S.
  - Interconexión mediante un bus.
  - Sistema operativo integrado para coordinación.
- Ventajas:
  - Mayor rendimiento si las tareas se paralelizan.
  - Escalabilidad incremental: Se pueden agregar más procesadores.
- Desventajas:
  - El bus compartido limita las prestaciones.
  - Problemas de coherencia de caché.

Arquitectura de un SMP



Cluster vs SMP

- Ambos:
  - dan soporte a aplicaciones de alta demanda de recursos
  - disponibles comercialmente (SMP es mas antiguo)
- SMP:
  - Mas fácil de administrar y configurar
  - Cercano a los sistemas de un solo procesador
    - La planificación (scheduling) es la diferencia principal
    - Menos espacio físico / Menor consumo de potencia
- Cluster:
  - Superior escalabilidad incremental y absoluta
  - Superior disponibilidad
    - Redundancia

Clusters:

- Conjunto de computadoras completas interconectadas que actúan como una sola máquina.

UMA, NUMA y CC-NUMA

- UMA (Uniform Memory Access):
  - Todos los procesadores tienen acceso uniforme a toda la memoria.
  - El tiempo de acceso a memoria es igual para todas las regiones y procesadores.
- NUMA (Non-Uniform Memory Access):
  - El tiempo de acceso a memoria varía según la región que se accede.
  - Permite usar más procesadores que los SMP tradicionales.
- CC-NUMA (Cache Coherent NUMA):
  - Un NUMA que mantiene la coherencia de caché entre los procesadores, asegurando consistencia en los datos.
  - Orden de acceso a memoria:
    1. Caché L1 (local al procesador).
    2. Caché L2 (local al procesador).
    3. Memoria principal (local al nodo).
    4. Memoria remota (vía red).

PROCESAMIENTO MULTIHEBRA (MULTITHREADING)

- Divide una secuencia de instrucciones en hebras (threads) que pueden ejecutarse en paralelo.

- Tipos:
  a. Multihebra explícito:
    - Hebras concurrentes de diferentes procesos.
    - Usado en todos los procesadores comerciales.
  b. Multihebra implícito:
    - Hebras extraídas de un único programa.
    - Generadas por el compilador o el hardware.
- Procesador multihebra:
  - Un contador de programa (PC) por hebra.
  - Utiliza predicción de saltos, renombre de registros, etc., para optimizar la ejecución.

ARQUITECTURAS "ON CHIP"

- Memoria compartida y distribuida integradas en procesadores multinúcleo.
- Optimización para reducir latencias en el acceso a memoria y mejorar la coordinación entre núcleos.

Preguntas de finales con respuestas

¿Que métodos para pasaje de argumento podemos utilizar en una computadora?

LOS MÉTODOS DE PASAJE DE ARGUMENTOS QUE SE PUEDEN UTILIZAR EN UNA COMPUTADORA INCLUYEN:

- VÍA REGISTROS: LOS ARGUMENTOS SE PASAN UTILIZANDO REGISTROS DEL PROCESADOR. ESTE MÉTODO ES RÁPIDO PERO LIMITADO POR EL NÚMERO DE REGISTROS DISPONIBLES. ES IMPORTANTE DOCUMENTAR QUÉ REGISTROS SE UTILIZAN.
- VÍA MEMORIA: LOS ARGUMENTOS SE ALMACENAN EN UN ÁREA DEFINIDA DE MEMORIA (RAM). ESTE MÉTODO NO DEPENDE DE LOS REGISTROS, PERO ES MÁS DIFÍCIL DE ESTANDARIZAR.
- VÍA PILA (STACK): ES EL MÉTODO MÁS AMPLIAMENTE USADO. LA PILA ES INDEPENDIENTE DE LA MEMORIA Y LOS REGISTROS. ESTE MÉTODO REQUIERE COMPRENDER BIEN EL FUNCIONAMIENTO DE LA PILA, YA QUE ES UTILIZADA TANTO POR EL USUARIO COMO POR EL SISTEMA.

¿Cuales son las diferencias en la terminación de una subrutina y un gestor de interrupción?

LA DIFERENCIA PRINCIPAL ENTRE LA TERMINACIÓN DE UNA SUBRUTINA Y UN GESTOR DE INTERRUPCIÓN ES CÓMO SE LLAMAN Y CÓMO TERMINAN. UNA SUBRUTINA SE LLAMA DIRECTAMENTE DESDE EL PROGRAMA PRINCIPAL CON UNA INSTRUCCIÓN (CALL) Y TERMINA DEVOLVIENDO EL CONTROL AL PROGRAMA CON RET.
EN CAMBIO, UN GESTOR DE INTERRUPCIÓN NO SE LLAMA DIRECTAMENTE. SE ACTIVA SOLO CUANDO OCURRE UN EVENTO ESPECIAL (COMO PRESIONAR UNA TECLA O RECIBIR DATOS). DESPUÉS DE EJECUTARSE, USA IRET PARA VOLVER AL PROGRAMA PRINCIPAL, ASEGURÁNDOSE DE QUE TODO SIGA IGUAL QUE ANTES DE LA INTERRUPCIÓN.
EN RESUMEN:

- LA SUBRUTINA SE USA CUANDO EL PROGRAMA NECESITA EJECUTAR ALGO ESPECÍFICO Y REGRESA NORMALMENTE AL PROGRAMA PRINCIPAL.
- EL GESTOR DE INTERRUPCIÓN RESPONDE A ALGO INESPERADO Y SE ASEGURA DE QUE EL PROGRAMA SIGA FUNCIONANDO BIEN DESPUÉS.

¿La coherencia de datos de un sistema jerárquico de memoria se ve afectado por el uso de DMA?

SÍ, EL USO DE DMA (ACCESO DIRECTO A MEMORIA) PUEDE AFECTAR LA COHERENCIA DE DATOS EN UN SISTEMA JERÁRQUICO DE MEMORIA. ESTO SE DEBE A QUE EL DMA PERMITE QUE DISPOSITIVOS PERIFÉRICOS ACCEDAN DIRECTAMENTE A LA MEMORIA PRINCIPAL SIN PASAR POR LA JERARQUÍA DE CACHÉ, LO QUE PUEDE DAR LUGAR A INCONSISTENCIAS DE DATOS EN LA CACHÉ Y LA MEMORIA PRINCIPAL. PARA MANTENER LA COHERENCIA DE DATOS, SE REQUIEREN TÉCNICAS ESPECIALES COMO INVALIDACIÓN DE CACHÉ Y PROTOCOLOS DE COHERENCIA DE CACHÉ PARA GARANTIZAR QUE LOS DATOS SE SINCRONICEN CORRECTAMENTE.

Describa cómo funciona la gestión de E/S programada con espera de respuesta.

EN ESTE MÉTODO, LA CPU CONTROLA DIRECTAMENTE LA OPERACIÓN DE ENTRADA/SALIDA A TRAVÉS DE LOS SIGUIENTES PASOS:

1. LA CPU SOLICITA UNA OPERACIÓN AL MÓDULO DE E/S (POR EJEMPLO, LEER O ESCRIBIR DATOS).
2. EL MÓDULO DE E/S REALIZA LA OPERACIÓN Y ACTUALIZA SU REGISTRO DE ESTADO, INDICANDO SI EL DISPOSITIVO ESTÁ LISTO O SI HUBO ALGÚN ERROR.
3. LA CPU VERIFICA CONSTANTEMENTE ESTE REGISTRO EN UN CICLO DE ESPERA ACTIVA ("BUSY WAITING") HASTA QUE EL DISPOSITIVO INDIQUE QUE HA COMPLETADO LA OPERACIÓN.
4. UNA VEZ QUE EL DISPOSITIVO COMPLETA LA OPERACIÓN, LA CPU TRANSFIERE O PROCESA LOS DATOS.

VENTAJAS:

- ES FÁCIL DE IMPLEMENTAR EN SISTEMAS SIMPLES.
- NO REQUIERE HARDWARE ADICIONAL COMO CONTROLADORES AVANZADOS DE INTERRUPCIONES O DMA.

DESVENTAJAS:

- LA CPU QUEDA INACTIVA DURANTE LA ESPERA, LO QUE GENERA UNA GRAN PÉRDIDA DE RENDIMIENTO.
- NO ES EFICIENTE PARA SISTEMAS MULTITAREA O CON MÚLTIPLES DISPOSITIVOS.
- ESCALA MAL EN SISTEMAS MODERNOS CON DISPOSITIVOS MÁS LENTOS.

ESTE MÉTODO ES ADECUADO ÚNICAMENTE PARA OPERACIONES MUY SIMPLES O DISPOSITIVOS QUE PROCESEN DATOS A VELOCIDADES SIMILARES A LAS DE LA CPU. SIN EMBARGO, SU USO ESTÁ LIMITADO EN SISTEMAS ACTUALES DEBIDO A SU INEFICIENCIA.

¿Cuál es el objetivo de usar la técnica de Renombre de Registros en un procesador superescalar?

EL OBJETIVO PRINCIPAL DE LA TÉCNICA DE RENOMBRE DE REGISTROS ES ELIMINAR LAS DEPENDENCIAS DE SALIDA (WAW) Y LAS ANTIDEPENDENCIAS (WAR) QUE IMPIDEN QUE MÚLTIPLES INSTRUCCIONES SE EJECUTEN SIMULTÁNEAMENTE. ESTO AUMENTA EL PARALELISMO Y MEJORA EL RENDIMIENTO DEL PROCESADOR.

LA TÉCNICA CONSISTE EN DISPONER DE REGISTROS ADICIONALES (INTERNOS Y OCULTOS AL PROGRAMADOR) QUE SON ASIGNADOS DINÁMICAMENTE POR EL HARDWARE A INSTRUCCIONES EN CONFLICTO. DE ESTE MODO, SE EVITA DETENER EL CAUCE, MAXIMIZANDO EL USO DE LAS UNIDADES FUNCIONALES Y DEJANDO SOLO LAS DEPENDENCIAS VERDADERAS (RAW), QUE SON INEVITABLES.

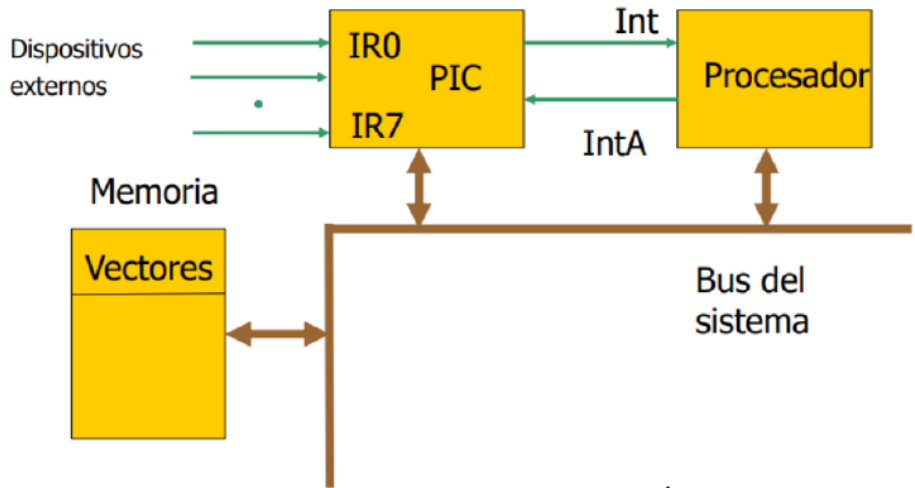
¿Qué son los MIMD de la taxonomía de Flynn?

LOS MIMD (MÚLTIPLES SECUENCIAS DE INSTRUCCIONES Y MÚLTIPLES SECUENCIAS DE DATOS) SON UNA DE LAS CATEGORÍAS DE COMPUTADORAS NOMBRADAS EN LA TAXONOMÍA DE FLYNN QUE HACE REFERENCIA AL CONJUNTO DE PROCESADORES QUE EJECUTAN SIMULTÁNEAMENTE SECUENCIAS DE INSTRUCCIONES DIFERENTES CON CONJUNTOS DE DATOS DIFERENTES. ESTO SIGNIFICA QUE SON CAPACES DE PROCESAR MÚLTIPLES INSTRUCCIONES AL MISMO TIEMPO Y UTILIZAR MÚLTIPLES DATOS EN PARALELO. HAY MÚLTIPLES UNIDADES DE CONTROL, Y CADA UNA PROPORCIONA UNA SECUENCIA DE INSTRUCCIONES SEPARADA A SU PROPIO ELEMENTO DE PROCESO.

EL MIMD PUEDE SER UN MULTIPROCESADOR DE MEMORIA COMPARTIDA (SMP O SISTEMAS NUMA), O UN MULTICOMPUTADOR DE MEMORIA DISTRIBUIDA (CLUSTERS).

Esquematice y describa la estructura interna de un Controlador Programable de Interrupciones.

EL PIC (CONTROLADOR DE INTERRUPCIONES PROGRAMABLE) ES UN DISPOSITIVO INTERNO CON LÍNEAS DE INTERRUPCIÓN POR HARDWARE, DONDE SE CONECTAN VARIAS DISPOSITIVOS QUE PUEDEN INTERRUMPIR A LA CPU. EL PIC PERMITE UTILIZAR VARIOS DISPOSITIVOS, MULTIPLEXANDO LOS PEDIDOS DE TODOS ELLOS A LA ÚNICA LÍNEA DE INTERRUPCIÓN DEL PROCESADOR.



EL PIC ESTÁ CONFORMADO POR VARIOS REGISTROS Y POR UNA LÓGICA DE CONTROL Y OTRA DE GESTIÓN DE PRIORIDAD, TODO CONECTADO POR UN BUS INTERNO DEL MISMO, LA COMUNICACIÓN ENTRE EL PIC Y EL PROCESADOR ES POR MEDIO DE SEÑALES Y COMPARTEN EL BUS DEL SISTEMA. LOS REGISTROS INTERNOS SON:

- ISR (INTERRUPCIÓN EN SERVICIO): SUS BITS INDICAN SI SE ESTÁ ATENDIENDO LA INTERRUPCIÓN DE ALGÚN DISPOSITIVO.
- IRR (INTERRUPCIONES PEDIDAS): SUS BITS INDICAN QUE DISPOSITIVOS ESTÁN SOLICITANDO UNA INTERRUPCIÓN.
- IMR (MÁSCARA DE INTERRUPCIONES): SUS BITS INDICAN QUÉ LÍNEAS ESTÁN HABILITADAS. PERMITE EL ENMASCARAMIENTO DE INTERRUPCIONES.
- EOI (FIN DE LA INTERRUPCIÓN): SUS BITS LE INDICAN AL PIC QUE LA INTERRUPCIÓN YA TERMINÓ Y FUE ATENDIDA.
- INTO A INT7: 8 REGISTROS, DONDE SE CARGAN LOS IDS DE LAS INTERRUPCIONES CORRESPONDIENTES.

¿De qué depende el paralelismo de una máquina superescalar?

EL PARALELISMO ES LA CAPACIDAD PARA EJECUTAR VARIAS TAREAS EN EL MISMO INTERVALO DE TIEMPO. EN UNA MÁQUINA SUPERESCALAR, ESTE DEPENDE TANTO A NIVEL DE INSTRUCCIONES COMO A NIVEL DE LA MÁQUINA.

- A NIVEL DE INSTRUCCIONES: ESTÁ LIMITADO POR LA FRECUENCIA DE DEPENDENCIAS DE DATOS VERDADERAS Y DEPENDENCIAS RELATIVAS AL PROCEDIMIENTO PRESENTES EN EL CÓDIGO. ESTOS FACTORES ESTÁN INFLUIDOS POR LA ARQUITECTURA DEL REPERTORIO DE INSTRUCCIONES Y LA NATURALEZA DE LA APLICACIÓN.
- A NIVEL DE LA MÁQUINA: DEPENDE DEL NÚMERO DE INSTRUCCIONES QUE PUEDEN CAPTARSE Y EJECUTARSE SIMULTÁNEAMENTE, DEL TAMAÑO DE LA VENTANA DE INSTRUCCIONES, DE LA CANTIDAD DE UNIDADES FUNCIONALES DISPONIBLES, Y DE LOS MECANISMOS COMO EL RENOMBRE DE REGISTROS Y LA EMISIÓN DESORDENADA QUE USA EL PROCESADOR PARA LOCALIZAR Y EJECUTAR INSTRUCCIONES INDEPENDIENTES.

¿Cuáles son los elementos a tener en cuenta para el diseño de una memoria Cache?

ELEMENTOS CLAVE PARA EL DISEÑO DE UNA MEMORIA CACHÉ

1. ORGANIZACIÓN:
   - TAMAÑO: DETERMINA LA CAPACIDAD DE LA CACHÉ, LIMITADA POR LAS RESTRICCIONES FÍSICAS DEL CHIP.
   - TAMAÑO DE LÍNEA: BLOQUES MÁS GRANDES MEJORAN LA LOCALIDAD ESPACIAL PERO REDUCEN LA CANTIDAD TOTAL DE BLOQUES.
   - NIVELES: PUEDE HABER UNA O MÁS CACHÉS (L1, L2, L3), UNIFICADAS O SEPARADAS PARA DATOS E INSTRUCCIONES.
2. FUNCIÓN DE CORRESPONDENCIA:

   DEFINE CÓMO SE ASIGNAN LOS BLOQUES DE MEMORIA PRINCIPAL A LA CACHÉ:
   - DIRECTA: SIMPLE PERO CON MAYOR TASA DE FALLOS EN ALGUNOS CASOS.
   - TOTALMENTE ASOCIATIVA: FLEXIBLE PERO MÁS COSTOSA.
   - ASOCIATIVA POR CONJUNTOS: EQUILIBRIO ENTRE FLEXIBILIDAD Y COSTO.
3. POLÍTICAS DE REEMPLAZO:

   DECIDEN QUÉ BLOQUE ELIMINAR AL CARGAR UNO NUEVO. ALGORITMOS COMUNES: LRU (MENOS RECIENTEMENTE USADO), FIFO (PRIMERO EN ENTRAR), Y LFU (MENOS FRECUENTEMENTE USADO).
4. POLÍTICAS DE ESCRITURA:
   - WRITE-THROUGH: ACTUALIZA SIMULTÁNEAMENTE CACHÉ Y MEMORIA PRINCIPAL, ASEGURANDO COHERENCIA PERO CON MAYOR TRÁFICO.
   - WRITE-BACK: SOLO ACTUALIZA LA CACHÉ Y SINCRONIZA CON LA MEMORIA EN REEMPLAZOS.
5. OPTIMIZACIÓN DEL ACCESO:

   APROVECHAR LA LOCALIDAD TEMPORAL Y ESPACIAL PARA MINIMIZAR FALLOS, REDUCIR EL TIEMPO DE ACIERTO Y LA PENALIZACIÓN POR FALLOS.

¿Qué elementos característicos definen un bus?

HAY ALGUNOS ELEMENTOS DE DISEÑO QUE SIRVEN PARA DEFINIR Y CARACTERIZAR A LOS BUSES:

- TIPO: LAS LÍNEAS DE BUS SE PUEDEN DIVIDIR EN DOS TIPOS GENÉRICOS: DEDICADAS Y MULTIPLEXADAS.
- MÉTODO DE ARBITRAJE: SE REFIERE AL PROCESO UTILIZADO PARA GESTIONAR Y CONTROLAR EL ACCESO DE MÚLTIPLES DISPOSITIVOS AL BUS (CENTRALIZADO O DISTRIBUIDO).
- TEMPORIZACIÓN: HACE REFERENCIA A LA COORDINACIÓN DE LAS OPERACIONES DE TRANSFERENCIA DE DATOS EN EL BUS EN RELACIÓN CON EL TIEMPO (SÍNCRONO O ASÍNCRONO) Y LA VELOCIDAD DEL RELOJ.
- ANCHURA DEL BUS: NÚMERO DE LÍNEAS O CABLES EN EL BUS QUE SE UTILIZAN PARA TRANSMITIR DATOS Y REFERENCIAS SIMULTÁNEAMENTE.
- TIPO DE TRANSFERENCIA DE DATOS: SE REFIERE AL MOVIMIENTO DE INFORMACIÓN DESDE UNA UBICACIÓN DE ALMACENAMIENTO HACIA OTRO COMPONENTE DEL SISTEMA (LECTURA O ESCRITURA) EN BLOQUES Y A SU LATENCIA.

Describa las características funcionales del acceso directo a memoria - DMA.

CARACTERÍSTICAS FUNCIONALES DEL DMA:

1. PERMITE TRANSFERENCIAS DE DATOS ENTRE PERIFÉRICOS Y MEMORIA DIRECTAMENTE, SIN INTERVENCIÓN CONSTANTE DE LA CPU.
2. EL DMAC ACTÚA COMO MAESTRO DEL BUS DURANTE LA TRANSFERENCIA, CONTROLANDO LA DIRECCIÓN DE MEMORIA Y LAS SEÑALES DE CONTROL.
3. LA CPU INICIALIZA EL PROCESO CONFIGURANDO PARÁMETROS COMO DIRECCIÓN, NÚMERO DE BYTES Y TIPO DE OPERACIÓN (LECTURA/ESCRITURA).
4. MODOS DE TRANSFERENCIA:
   - RÁFAGAS: COMPLETA TODO EL BLOQUE DE DATOS EN UNA SOLA OPERACIÓN (RÁPIDO PERO BLOQUEA EL BUS PARA LA CPU).
   - ROBO DE CICLO: TRANSFIERE UNA PALABRA A LA VEZ, PERMITIENDO QUE LA CPU UTILICE EL BUS ENTRE TRANSFERENCIAS (MÁS LENTO PERO EFICIENTE).
5. AL FINALIZAR, EL DMAC NOTIFICA A LA CPU MEDIANTE UNA INTERRUPCIÓN, LIBERÁNDOLA PARA OTRAS TAREAS DURANTE LA OPERACIÓN.

EN RESUMEN: EL DMA PERMITE TRANSFERIR DATOS ENTRE MEMORIA Y PERIFÉRICOS DE FORMA AUTÓNOMA, LIBERANDO A LA CPU DE ESTA TAREA Y AUMENTANDO LA EFICIENCIA DEL SISTEMA.

Explique el mecanismo de interrupción

EL MECANISMO DE INTERRUPCIÓN ES UN PROCESO QUE PERMITE INTERRUMPIR LA EJECUCIÓN NORMAL DEL PROGRAMA EN LA CPU PARA ATENDER EVENTOS EXTERNOS O INTERNOS. ESTE MECANISMO OPERA DE LA SIGUIENTE MANERA:

1. DETECCIÓN DE LA INTERRUPCIÓN:
   - UN DISPOSITIVO EXTERNO O UNA CONDICIÓN INTERNA GENERA UNA SEÑAL DE INTERRUPCIÓN HACIA LA CPU.
   - LA CPU VERIFICA SI HAY SOLICITUDES DE INTERRUPCIÓN PENDIENTES AL FINAL DE CADA CICLO DE INSTRUCCIÓN.
2. PAUSA EN LA EJECUCIÓN DEL PROGRAMA:
   - SI SE DETECTA UNA INTERRUPCIÓN, LA CPU DETIENE TEMPORALMENTE LA EJECUCIÓN DEL PROGRAMA EN CURSO Y GUARDA SU ESTADO ACTUAL (REGISTRO DE INSTRUCCIONES Y OTROS PARÁMETROS IMPORTANTES) PARA PODER RETOMARLO MÁS TARDE.
3. LOCALIZACIÓN DE LA RUTINA DE SERVICIO:
   - EN EL CASO DE INTERRUPCIONES VECTORIZADAS, LA CPU UTILIZA UN VECTOR (PROPORCIONADO POR EL DISPOSITIVO O UN CONTROLADOR COMO EL PIC) PARA LOCALIZAR LA DIRECCIÓN EN MEMORIA DE LA RUTINA DE SERVICIO ASOCIADA AL EVENTO.
4. ATENCIÓN DE LA INTERRUPCIÓN:
   - LA CPU TRANSFIERE EL CONTROL A LA RUTINA DE SERVICIO, QUE EJECUTA LAS INSTRUCCIONES NECESARIAS PARA MANEJAR EL EVENTO QUE GENERÓ LA INTERRUPCIÓN.
5. RETORNO AL PROGRAMA PRINCIPAL:
   - AL FINALIZAR LA RUTINA DE SERVICIO, LA CPU RESTAURA EL ESTADO DEL PROGRAMA PRINCIPAL Y REANUDA SU EJECUCIÓN EXACTAMENTE DONDE FUE INTERRUMPIDO.

Describa cómo se realiza el reconocimiento de interrupciones vectorizadas mediante el PIC.

CUANDO UN DISPOSITIVO GENERA UNA INTERRUPCIÓN A TRAVÉS DE UNA LÍNEA DE SOLICITUD, EL PIC REALIZA LAS SIGUIENTES ACCIONES:

1. VERIFICACIÓN DE CONDICIONES:
   - COMPRUEBA QUE LA LÍNEA DE INTERRUPCIÓN NO ESTÉ ENMASCARADA (HABILITADA PARA GENERAR INTERRUPCIONES).
   - REVISA QUE NO HAYA OTROS DISPOSITIVOS CON MAYOR PRIORIDAD QUE TAMBIÉN QUIERAN INTERRUMPIR.
2. SOLICITUD DE INTERRUPCIÓN A LA CPU:
   - SI LAS VERIFICACIONES ANTERIORES SON EXITOSAS, EL PIC ENVÍA UNA SEÑAL DE SOLICITUD DE INTERRUPCIÓN (INTR) A LA CPU, SOLICITANDO SU ATENCIÓN.
   - LA CPU RESPONDE CON UNA SEÑAL DE RECONOCIMIENTO (INTA) CUANDO ESTÁ LISTA PARA ATENDER LA INTERRUPCIÓN.
3. GENERACIÓN DEL VECTOR DE INTERRUPCIÓN:
   - AL RECIBIR LA SEÑAL INTA, EL PIC COLOCA EL VECTOR DE INTERRUPCIÓN CORRESPONDIENTE EN EL BUS DE DATOS. ESTE VECTOR IDENTIFICA EL DISPOSITIVO O EVENTO QUE GENERÓ LA INTERRUPCIÓN.
4. GESTIÓN POR PARTE DE LA CPU:
   - LA CPU UTILIZA EL VECTOR PROPORCIONADO POR EL PIC PARA ACCEDER A LA TABLA DE VECTORES DE INTERRUPCIÓN, DONDE SE ENCUENTRA LA DIRECCIÓN DE LA RUTINA DE SERVICIO ASOCIADA.
   - A CONTINUACIÓN, LA CPU GUARDA EL CONTEXTO DEL PROGRAMA EN EJECUCIÓN, TRANSFIERE EL CONTROL A LA RUTINA DE SERVICIO Y GESTIONA LA INTERRUPCIÓN.
5. EJECUCIÓN Y RETORNO:
   - UNA VEZ FINALIZADA LA RUTINA DE SERVICIO, LA CPU RESTAURA EL CONTEXTO DEL PROGRAMA INTERRUMPIDO Y REANUDA SU EJECUCIÓN NORMAL.

Describa los posibles modos de ubicación de los módulos de E/S.

1. E/S ASIGNADA EN MEMORIA (MEMORY-MAPPED I/O):

- EN ESTE MODO, LOS DISPOSITIVOS DE E/S Y LA MEMORIA COMPARTEN UN ÚNICO ESPACIO DE DIRECCIONES.
- LOS DISPOSITIVOS DE E/S SE TRATAN COMO SI FUERAN UBICACIONES DE MEMORIA, Y LAS OPERACIONES DE LECTURA Y ESCRITURA SE REALIZAN UTILIZANDO INSTRUCCIONES DE ACCESO A MEMORIA.
- ESTE ENFOQUE PERMITE UNA PROGRAMACIÓN EFICIENTE DEBIDO A LA VARIEDAD DE INSTRUCCIONES DISPONIBLES PARA OPERAR SOBRE LOS DISPOSITIVOS.

2. E/S AISLADA:

- EN ESTE CASO, LOS DISPOSITIVOS DE E/S TIENEN UN ESPACIO DE DIRECCIONES SEPARADO DE LA MEMORIA.
- SE REQUIEREN LÍNEAS ESPECÍFICAS PARA SEPARAR LAS OPERACIONES DE E/S DE LAS OPERACIONES DE MEMORIA.
- ESTE MÉTODO UTILIZA INSTRUCCIONES DEDICADAS EXCLUSIVAMENTE PARA OPERACIONES DE E/S, LO QUE LIMITA LA VARIEDAD DE OPERACIONES DISPONIBLES EN COMPARACIÓN CON LA E/S ASIGNADA EN MEMORIA.

¿A que método de atención lo conocemos como de "interrupciones vectorizadas ?

LAS INTERRUPCIONES VECTORIZADAS SON UN MÉTODO EFICIENTE PARA MANEJAR INTERRUPCIONES EN SISTEMAS. CUANDO UN MÓDULO GENERA UNA INTERRUPCIÓN, ENVÍA UN VECTOR (UN IDENTIFICADOR) AL BUS DE DATOS. ESTE VECTOR PERMITE AL PROCESADOR O AL CONTROLADOR DE INTERRUPCIONES IDENTIFICAR DE FORMA DIRECTA TANTO LA RUTINA DE ATENCIÓN CORRESPONDIENTE COMO EL PERIFÉRICO QUE EMITIÓ LA INTERRUPCIÓN, ELIMINANDO LA NECESIDAD DE PREGUNTAR A CADA MÓDULO (POLLING). EL VECTOR PUEDE SER COLOCADO POR EL PERIFÉRICO O GESTIONADO POR UN CONTROLADOR DE INTERRUPCIONES PROGRAMABLE (PIC). ESTE ENFOQUE OPTIMIZA EL TIEMPO DE RESPUESTA Y SIMPLIFICA LA GESTIÓN DE INTERRUPCIONES EN SISTEMAS COMPLEJOS.

¿cuándo, porqué, para qué y cómo utiliza una de las denominadas interrupciones por software?

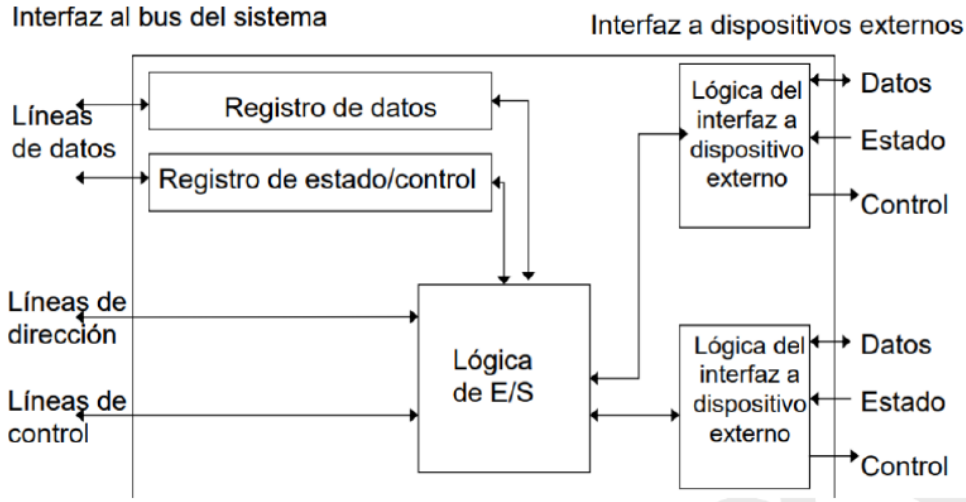
LAS INTERRUPCIONES POR SOFTWARE SE UTILIZAN PRINCIPALMENTE PARA REALIZAR LLAMADAS A FUNCIONES DEL SISTEMA OPERATIVO DE MANERA FLEXIBLE Y EFICIENTE. ESTAS INTERRUPCIONES NO DEPENDEN DE DISPOSITIVOS DE E/S FÍSICOS Y SE GENERAN MEDIANTE LA EJECUCIÓN DE UNA INSTRUCCIÓN ESPECÍFICA EN EL PROGRAMA, COMO INT XX. AL ACTIVARSE, INTERRUMPEN LA SECUENCIA NORMAL DE EJECUCIÓN Y TRANSFIEREN EL CONTROL AL MANEJADOR DE INTERRUPCIONES CORRESPONDIENTE, QUE EJECUTA LA TAREA ASOCIADA. ESTE MECANISMO PERMITE QUE LOS PROGRAMAS UTILICEN RUTINAS DEL SISTEMA OPERATIVO SIN NECESIDAD DE CONOCER SUS DIRECCIONES EXACTAS, OPTIMIZANDO LA INTERACCIÓN ENTRE EL SOFTWARE DE APLICACIÓN Y EL SISTEMA OPERATIVO.

Describa las funciones de correspondencia entre Memoria Principal y Cache

LA ELECCIÓN DE LA FUNCIÓN DE CORRESPONDENCIA DETERMINA CÓMO SE ORGANIZAN Y ASIGNAN LOS BLOQUES DE MEMORIA PRINCIPAL A LA CACHÉ. EXISTEN TRES TÉCNICAS PRINCIPALES:

- CORRESPONDENCIA DIRECTA: CADA BLOQUE DE MEMORIA PRINCIPAL SE ASIGNA A UNA LÍNEA ESPECÍFICA DE LA CACHÉ, CALCULADA MEDIANTE UNA FUNCIÓN MODULAR. ES SENCILLA Y ECONÓMICA DE IMPLEMENTAR, PERO PUEDE GENERAR UNA ALTA TASA DE CONFLICTOS SI VARIOS BLOQUES COMPARTEN LA MISMA LÍNEA.
- CORRESPONDENCIA ASOCIATIVA: UN BLOQUE DE MEMORIA PRINCIPAL PUEDE ALMACENARSE EN CUALQUIER LÍNEA DE LA CACHÉ. PARA LOCALIZARLO, SE UTILIZA UNA ETIQUETA QUE IDENTIFICA EL BLOQUE EN MEMORIA PRINCIPAL. OFRECE MAYOR FLEXIBILIDAD Y REDUCE LOS CONFLICTOS, PERO AUMENTA LOS COSTOS DE HARDWARE DEBIDO A LA NECESIDAD DE BUSCAR SIMULTÁNEAMENTE EN TODAS LAS LÍNEAS.
- CORRESPONDENCIA ASOCIATIVA POR CONJUNTOS: COMBINA LAS VENTAJAS DE LOS OTROS MÉTODOS. LA CACHÉ SE DIVIDE EN CONJUNTOS, Y UN BLOQUE DE MEMORIA PRINCIPAL PUEDE ALMACENARSE EN CUALQUIER LÍNEA DENTRO DE UN CONJUNTO ESPECÍFICO. ESTO REDUCE LOS CONFLICTOS EN COMPARACIÓN CON LA CORRESPONDENCIA DIRECTA, MANTENIENDO UN COSTO MÁS BAJO QUE LA CORRESPONDENCIA TOTALMENTE ASOCIATIVA.

Esquematice y describa la estructura interna de un módulo de E/S



1. INTERFAZ AL BUS DEL SISTEMA:

- LÍNEAS DE DATOS:
  - CANAL POR DONDE SE TRANSFIEREN LOS DATOS ENTRE EL BUS DEL SISTEMA Y EL MÓDULO DE E/S.
- LÍNEAS DE DIRECCIÓN:
  - IDENTIFICAN EL DISPOSITIVO O EL REGISTRO ESPECÍFICO EN EL MÓDULO AL QUE SE DEBE ACCEDER.
- LÍNEAS DE CONTROL:
  - GESTIONAN LAS OPERACIONES, COMO LECTURA, ESCRITURA O INTERRUPCIONES.

2. REGISTRO DE DATOS:

- ALMACENA TEMPORALMENTE LOS DATOS TRANSFERIDOS ENTRE EL SISTEMA Y EL DISPOSITIVO EXTERNO.

3. REGISTRO DE ESTADO/CONTROL:

- INDICA EL ESTADO DEL DISPOSITIVO (POR EJEMPLO, SI ESTÁ LISTO PARA TRANSMITIR O RECIBIR).
- PERMITE CONFIGURAR O ENVIAR ÓRDENES DE CONTROL AL DISPOSITIVO.

4. LÓGICA DE E/S:

- ES EL NÚCLEO DEL MÓDULO DE E/S. COORDINA LAS SEÑALES ENTRE EL BUS DEL SISTEMA Y LOS DISPOSITIVOS EXTERNOS.

5. INTERFAZ A DISPOSITIVOS EXTERNOS:

- LÓGICA DEL INTERFAZ A DISPOSITIVO EXTERNO:
  - ADAPTA LAS SEÑALES ELÉCTRICAS Y LOS PROTOCOLOS NECESARIOS PARA COMUNICARSE CON LOS DISPOSITIVOS EXTERNOS.
- MANEJA LAS SEÑALES DE DATOS, ESTADO Y CONTROL ENTRE EL MÓDULO Y LOS PERIFÉRICOS.

¿Qué es un riesgo WAR en un cauce segmentado?

UN RIESGO WAR (WRITE AFTER READ O ANTIDEPENDENCIA) OCURRE EN UN CAUCE SEGMENTADO CUANDO UNA INSTRUCCIÓN POSTERIOR INTENTA ESCRIBIR EN UN REGISTRO ANTES DE QUE UNA INSTRUCCIÓN ANTERIOR HAYA TERMINADO DE LEER ESE MISMO REGISTRO. ESTO PUEDE CAUSAR QUE LA LECTURA OBTENGA UN VALOR INCORRECTO DEBIDO AL ADELANTAMIENTO DE LA ESCRITURA.

SIN EMBARGO, EN UN CAUCE SEGMENTADO BÁSICO, LOS RIESGOS WAR NO OCURREN PORQUE LAS INSTRUCCIONES ESTÁN DISEÑADAS PARA QUE LAS LECTURAS SE COMPLETEN ANTES DE QUE SE REALICEN LAS ESCRITURAS, GARANTIZANDO QUE NO HAYA CONFLICTOS POR ANTIDEPENDENCIAS.

¿Qué es la segmentación del cauce de instrucciones? ¿Cuánto mejora el rendimiento?

LA SEGMENTACIÓN DE CAUCE (O PIPELINING) ES UNA TÉCNICA DE DISEÑO DE HARDWARE QUE ORGANIZA LA CPU PARA EJECUTAR VARIAS OPERACIONES SIMULTÁNEAMENTE. CONSISTE EN DIVIDIR EL PROCESO DE EJECUCIÓN DE INSTRUCCIONES EN FASES O ETAPAS, PERMITIENDO QUE ESTAS SE EJECUTEN EN PARALELO.
EL MÁXIMO RENDIMIENTO EN UN PROCESADOR SEGMENTADO ES COMPLETAR UNA INSTRUCCIÓN EN CADA CICLO DE RELOJ.
LA VELOCIDAD DE UN PROCESADOR SEGMENTADO PUEDE APROXIMARSE MEDIANTE LA FÓRMULA:

$$\text{Vel. procesador segmentado} = \text{Vel. secuencial} \times K$$

DONDE K ES EL NÚMERO DE ETAPAS DEL CAUCE. ESTO IMPLICA QUE EL INCREMENTO POTENCIAL EN EL RENDIMIENTO ES PROPORCIONAL AL NÚMERO DE ETAPAS DEL CAUCE

¿Porqué funciona un sistema de memoria basado en jerarquía?

UN SISTEMA DE MEMORIA BASADO EN JERARQUÍA FUNCIONA GRACIAS AL PRINCIPIO DE LOCALIDAD DE REFERENCIAS, QUE SE DIVIDE EN DOS ASPECTOS PRINCIPALES:

1. LOCALIDAD TEMPORAL: LOS ELEMENTOS DE MEMORIA QUE HAN SIDO REFERENCIADOS RECIENTEMENTE TIENEN ALTA PROBABILIDAD DE SER ACCEDIDOS NUEVAMENTE EN UN FUTURO CERCANO. POR LO TANTO, ES EFICIENTE MANTENER ESOS ELEMENTOS EN UN NIVEL DE MEMORIA MÁS RÁPIDO.
2. LOCALIDAD ESPACIAL: LOS ELEMENTOS DE MEMORIA CUYAS DIRECCIONES ESTÁN CERCA DE LOS ELEMENTOS RECIENTEMENTE REFERENCIADOS TAMBIÉN TIENEN ALTA PROBABILIDAD DE SER ACCEDIDOS PRONTO. POR ESTO, AL MOVER UN DATO A UN NIVEL SUPERIOR DE LA JERARQUÍA, SE SUELE TRANSFERIR UN BLOQUE COMPLETO QUE INCLUYE ESE DATO.

ESTA ESTRUCTURA JERÁRQUICA PERMITE QUE EL SISTEMA TENGA LA VELOCIDAD DEL NIVEL MÁS RÁPIDO AL COSTO DEL NIVEL MÁS BARATO, YA QUE LOS DATOS MÁS FRECUENTEMENTE UTILIZADOS ESTÁN EN NIVELES SUPERIORES (MÁS RÁPIDOS Y PEQUEÑOS), MIENTRAS QUE LOS MENOS USADOS PERMANECEN EN NIVELES INFERIORES (MÁS LENTOS Y GRANDES).

¿Qué es y de qué depende el paralelismo de una máquina?

EL PARALELISMO DE UNA MÁQUINA SE DEFINE COMO UNA MEDIDA DE LA CAPACIDAD DEL PROCESADOR PARA APROVECHAR EL PARALELISMO ENTRE INSTRUCCIONES, LO QUE IMPLICA EJECUTAR VARIAS INSTRUCCIONES EN PARALELO.
EL GRADO DE PARALELISMO DE UNA MÁQUINA DEPENDE DE LOS SIGUIENTES FACTORES:

1. NÚMERO DE INSTRUCCIONES CAPTADAS POR CICLO: CUANTAS MÁS INSTRUCCIONES SE PUEDAN CAPTAR SIMULTÁNEAMENTE, MAYOR SERÁ EL POTENCIAL DE PARALELISMO.
2. NÚMERO DE UNIDADES FUNCIONALES: LA PRESENCIA DE MÚLTIPLES UNIDADES PERMITE QUE VARIAS INSTRUCCIONES SE EJECUTEN EN PARALELO.
3. MECANISMOS PARA LOCALIZAR INSTRUCCIONES INDEPENDIENTES: ESTOS INCLUYEN LA IDENTIFICACIÓN DE PARALELISMO EN LAS INSTRUCCIONES Y LA ORGANIZACIÓN DE LAS ETAPAS DE BÚSQUEDA, DECODIFICACIÓN Y EJECUCIÓN EN PARALELO.
4. RENOMBRE DE REGISTROS: ESTO PERMITE RESOLVER DEPENDENCIAS DE SALIDA Y ANTIDEPENDENCIAS, REDUCIENDO CONFLICTOS EN EL USO DE REGISTROS.
5. VENTANA DE INSTRUCCIONES: HABILITA LA EMISIÓN DESORDENADA DE INSTRUCCIONES PARA MAXIMIZAR EL USO DE LOS RECURSOS DISPONIBLES.

Analice las políticas de escritura desde el punto de vista de la coherencia de datos

DESDE EL PUNTO DE VISTA DE LA COHERENCIA DE DATOS, ES FUNDAMENTAL EVITAR INCONSISTENCIAS ENTRE LA MEMORIA CACHÉ Y LA MEMORIA PRINCIPAL, ESPECIALMENTE DURANTE LAS OPERACIONES DE ESCRITURA. AUNQUE LAS ESCRITURAS SE REALIZAN INICIALMENTE EN LA CACHÉ, EN ALGÚN MOMENTO EL BLOQUE CORRESPONDIENTE DE LA MEMORIA PRINCIPAL DEBE SER ACTUALIZADO.

LAS POLÍTICAS COMUNES SON LAS SIGUIENTES:

1. EN ACIERTOS:
   - ESCRITURA INMEDIATA (WRITE-THROUGH): TODAS LAS ESCRITURAS SE REALIZAN TANTO EN LA CACHÉ COMO EN LA MEMORIA PRINCIPAL. ESTO GARANTIZA UNA ALTA COHERENCIA DE DATOS, PERO AUMENTA EL TRÁFICO EN EL BUS DEL SISTEMA.
   - POST ESCRITURA (WRITE-BACK): LAS ESCRITURAS SE REALIZAN ÚNICAMENTE EN LA CACHÉ, Y LA MEMORIA PRINCIPAL SE ACTUALIZA CUANDO EL BLOQUE ES REEMPLAZADO. ESTA POLÍTICA MEJORA EL RENDIMIENTO, PERO PUEDE GENERAR PROBLEMAS DE COHERENCIA SI NO SE EMPLEAN PROTOCOLOS ADICIONALES.
2. EN FALLOS:
   - WRITE ALLOCATE: CUANDO SE ESCRIBE UN DATO Y EL BLOQUE NO ESTÁ EN LA CACHÉ, PRIMERO SE COPIA EL BLOQUE DE LA MEMORIA PRINCIPAL A LA CACHÉ, Y LUEGO SE REALIZA LA ESCRITURA. ESTO PERMITE QUE LAS FUTURAS ESCRITURAS EN EL MISMO BLOQUE SEAN MÁS EFICIENTES.
   - NO-WRITE ALLOCATE: EN ESTE CASO, EL DATO SE ESCRIBE DIRECTAMENTE EN LA MEMORIA PRINCIPAL SIN COPIAR EL BLOQUE A LA CACHÉ. AUNQUE REDUCE LA COMPLEJIDAD, PUEDE SER MENOS EFICIENTE.

¿ Que características tienen los procesadores superescalares? Describa las causas que pueden retardar el funcionamiento de los mismos

LOS PROCESADORES SUPERESCALARES SON AQUELLOS DISEÑADOS PARA EJECUTAR MÚLTIPLES INSTRUCCIONES DE MANERA SIMULTÁNEA E INDEPENDIENTE, INCREMENTANDO EL PARALELISMO EN EL FLUJO DE TRABAJO. SUS PRINCIPALES CARACTERÍSTICAS INCLUYEN:

- CAPACIDAD PARA EJECUTAR DOS O MÁS INSTRUCCIONES POR CICLO DE RELOJ, GRACIAS A ESTRATEGIAS DE PARALELISMO.
- MÚLTIPLES UNIDADES FUNCIONALES INDEPENDIENTES SEGMENTADAS QUE PERMITEN EJECUTAR OPERACIONES EN PARALELO.
- JERARQUÍAS DE MEMORIA OPTIMIZADAS PARA ATENDER MÚLTIPLES ACCESOS SIMULTÁNEAMENTE.
- LÓGICA AVANZADA PARA DETECTAR DEPENDENCIAS ENTRE REGISTROS, COMO DEPENDENCIAS VERDADERAS, DE SALIDA Y ANTIDEPENDENCIAS.
- ESTRATEGIAS DE CAPTACIÓN DE INSTRUCCIONES EN PARALELO, QUE MEJORAN EL FLUJO DE EJECUCIÓN.

EL FUNCIONAMIENTO DE LOS PROCESADORES SUPERESCALARES PUEDE VERSE LIMITADO POR DIVERSOS FACTORES, COMO:

- DEPENDENCIAS DE DATOS: RIESGOS COMO RAW, WAW Y WAR QUE DIFICULTAN EL PARALELISMO.
- DEPENDENCIAS DE CONTROL: SALTOS Y BIFURCACIONES QUE GENERAN INCERTIDUMBRE EN EL FLUJO DE EJECUCIÓN, MITIGADOS MEDIANTE PREDICCIÓN DE BIFURCACIONES.
- CONFLICTOS EN LOS RECURSOS: VARIAS INSTRUCCIONES COMPITIENDO POR LAS MISMAS UNIDADES FUNCIONALES.
- ANCHO DE BANDA DE MEMORIA: LIMITACIONES EN LA TRANSFERENCIA DE DATOS ENTRE MEMORIA Y PROCESADOR.
- LLAMADAS A SUBRUTINAS: INTRODUCEN RETRASOS ADICIONALES EN EL FLUJO.

Que elementos componen una máquina con arquitectura Von Neumann? Describir la función de cada uno

LOS ELEMENTOS QUE COMPONEN UNA MÁQUINA CON ARQUITECTURA VON NEUMANN

1. UNIDAD CENTRAL DE PROCESAMIENTO (CPU):
   - UNIDAD DE CONTROL (UC): COORDINA LAS OPERACIONES DEL SISTEMA, INTERPRETANDO LAS INSTRUCCIONES Y GENERANDO LAS SEÑALES NECESARIAS PARA EJECUTARLAS.
   - UNIDAD ARITMÉTICO-LÓGICA (ALU): REALIZA OPERACIONES MATEMÁTICAS (SUMA, RESTA, ETC.) Y LÓGICAS (AND, OR, NOT).
2. MEMORIA PRINCIPAL:
   - ALMACENA TEMPORALMENTE DATOS E INSTRUCCIONES QUE LA CPU NECESITA PARA EJECUTAR LOS PROGRAMAS.
3. COMPONENTES DE ENTRADA/SALIDA (E/S):
   - PERMITEN LA INTERACCIÓN ENTRE LA MÁQUINA Y EL USUARIO O DISPOSITIVOS EXTERNOS, COMO TECLADO, PANTALLA, IMPRESORAS, ETC.
4. REGISTROS INTERNOS DE LA CPU:
   - CONTADOR DE PROGRAMA (PC): CONTIENE LA DIRECCIÓN DE LA SIGUIENTE INSTRUCCIÓN A EJECUTAR.
   - REGISTRO DE INSTRUCCIÓN (IR): ALMACENA LA INSTRUCCIÓN ACTUAL QUE ESTÁ SIENDO EJECUTADA.
   - REGISTRO DE DIRECCIÓN DE MEMORIA (MAR): CONTIENE LA DIRECCIÓN DE LA MEMORIA QUE SE VA A LEER O ESCRIBIR.
   - REGISTRO DE BUFFER DE MEMORIA (MBR): ALMACENA LOS DATOS QUE SE TRANSFIEREN HACIA/DESDE LA MEMORIA.
   - REGISTROS DE E/S (AR Y BR): MANEJAN LAS DIRECCIONES Y DATOS DURANTE LAS OPERACIONES DE ENTRADA/SALIDA.

Compare los sistemas MP y clusters

SMP:

- SON MÁS FACILES DE GESTIONAR Y CONFIGURAR QUE UN CLUSTER.
- SON MÁS CERCANOS AL MODELO DE COMPUTADOR DE UN SOLO PROCESADOR, DISPONIBLE EN CASI TODAS LAS APLICACIONES.
- EL PRINCIPAL CAMBIO QUE SE NECESITA PARA PASAR DE UN COMPUTADOR MONOPROCESADOR A UN SMP SE REFIERE AL FUNCIONAMIENTO DEL PLANIFICADOR.
- NECESITAN MENOS ESPACIO FÍSICO Y CONSUMEN MENOS ENERGÍA QUE UN CLUSTER COMPARABLE.
- SON PLATAFORMAS ESTABLES Y BIEN ESTABLECIDAS.

CLUSTERS:

- SON SUPERIORES A LOS SMP EN TÉRMINOS DE ESCALABILIDAD ABSOLUTA E INCREMENTAL.
- SON SUPERIORES TAMBIÉN EN TÉRMINOS DE DISPONIBILIDAD, YA QUE TODOS LOS COMPONENTES DEL SISTEMA PUEDE HACER ALTAMENTE REDUNDANTES.
- LOS NODOS CUENTAN CADA UNO CON MEMORIA PRIVADA, Y LAS APLICACIONES NO VEN LA MEMORIA GLOBAL.

Describa el problema y posibles soluciones ante riesgos por transferencia de control del programa.

LOS RIESGOS POR TRANSFERENCIA DE CONTROL DEL PROGRAMA, ESPECIALMENTE EN INSTRUCCIONES DE SALTO (CONDICIONALES O INCONDICIONALES), GENERAN CONFLICTOS PORQUE EL PROCESADOR NO PUEDE DETERMINAR INMEDIATAMENTE EL PRÓXIMO PASO A EJECUTAR. ESTO CAUSA PARADAS (STALLS) EN EL FLUJO DEL CAUCE Y AFECTA LA EFICIENCIA GENERAL DEL SISTEMA.

POSIBLES SOLUCIONES:

1. POR HARDWARE:
   - PREDICCIÓN DE SALTOS:
     - ESTÁTICA: ASUME SIEMPRE EL MISMO COMPORTAMIENTO DEL SALTO (SIEMPRE SALTA O NUNCA SALTA).
     - DINÁMICA: UTILIZA UN HISTORIAL DE EJECUCIONES PREVIAS PARA TOMAR DECISIONES MÁS PRECISAS (EJ., CONMUTADORES O TABLAS DE HISTORIA DE SALTOS).
   - ADELANTAR RESOLUCIÓN DEL SALTO: RESOLVER EL SALTO EN ETAPAS TEMPRANAS DEL CAUCE (DECODIFICACIÓN) PARA MINIMIZAR RETRASOS.
   - CAUCES MÚLTIPLES: USAR VARIOS CAUCES PARA EVALUAR SIMULTÁNEAMENTE LAS OPCIONES DE SALTO Y CONTINUAR CON LA CORRECTA.
   - PRECAPTACIÓN DEL DESTINO DEL SALTO: CAPTURAR ANTICIPADAMENTE LAS INSTRUCCIONES EN AMBAS RAMAS DEL SALTO Y CONSERVAR LA ADECUADA.
   - BUFFER DE BUCLES: UTILIZAR UNA MEMORIA RÁPIDA QUE ALMACENE LAS INSTRUCCIONES DE PEQUEÑOS BUCLES PARA ACELERAR SU EJECUCIÓN.
2. POR SOFTWARE:
   - SALTO RETARDADO: EL COMPILADOR ORGANIZA LAS INSTRUCCIONES PARA INSERTAR TRABAJO ÚTIL EN LOS CICLOS DE PENALIZACIÓN TRAS EL SALTO.
   - REORDENAMIENTO DE INSTRUCCIONES: COLOCA INSTRUCCIONES NO DEPENDIENTES DEL SALTO EN POSICIONES ESTRATÉGICAS PARA EVITAR CONFLICTOS.

Describa tres diferentes causas que pueden retardar un cauce de instrucciones segmentado.¿Qué retardo produce cada una?

LAS CAUSAS PRINCIPALES QUE PUEDEN RETARDAR UN CAUCE SEGMENTADO SON:

1. RIESGOS ESTRUCTURALES:
   - SUCEDEN CUANDO DOS INSTRUCCIONES NECESITAN USAR EL MISMO RECURSO DE HARDWARE EN EL MISMO CICLO.
   - RESULTADO: LA INSTRUCCIÓN QUE LLEGA DESPUÉS DEBE ESPERAR (STALL), RETRASANDO EL CAUCE.
2. DEPENDENCIAS DE DATOS:
   - UNA INSTRUCCIÓN DEPENDE DE LOS DATOS GENERADOS POR OTRA QUE AÚN NO HA TERMINADO.
   - RESULTADO: SE INTRODUCE UN RETARDO HASTA QUE LOS DATOS ESTÉN DISPONIBLES.
3. DEPENDENCIAS DE CONTROL:
   - OCURREN CUANDO NO SE SABE CUÁL SERÁ LA SIGUIENTE INSTRUCCIÓN A EJECUTAR DEBIDO A UN SALTO.
   - RESULTADO: EL CAUCE SE DETIENE MIENTRAS SE RESUELVE EL SALTO Y SE ACTUALIZA EL PC.

Compare las políticas de emisión de instrucciones. Elija una alternativa de emisión/finalización y justifique.

POLÍTICAS DE EMISIÓN DE INSTRUCCIONES: LOS PROCESADORES SUPERESCALARES UTILIZAN DIFERENTES POLÍTICAS PARA EMITIR INSTRUCCIONES A SUS UNIDADES FUNCIONALES, LO QUE AFECTA EL PARALELISMO Y LA COMPLEJIDAD DEL HARDWARE. ESTAS POLÍTICAS SON:

1. EMISIÓN EN ORDEN Y FINALIZACIÓN EN ORDEN:
   - LAS INSTRUCCIONES SE EMITEN Y COMPLETAN EN EL MISMO ORDEN QUE APARECEN EN EL PROGRAMA.
   - VENTAJAS: SIMPLICIDAD EN EL HARDWARE.
   - DESVENTAJAS: BAJO PARALELISMO Y SUBUTILIZACIÓN DE LOS RECURSOS.
2. EMISIÓN EN ORDEN Y FINALIZACIÓN DESORDENADA:
   - LAS INSTRUCCIONES SE EMITEN EN ORDEN, PERO PUEDEN FINALIZAR EN CUALQUIER SECUENCIA.
   - VENTAJAS: MEJORA EL USO DE RECURSOS AL PERMITIR QUE LAS INSTRUCCIONES INDEPENDIENTES FINALICEN ANTES.
   - DESVENTAJAS: REQUIERE LÓGICA COMPLEJA PARA EVITAR CONFLICTOS Y MANTENER LA COHERENCIA DEL PROGRAMA.
3. EMISIÓN Y FINALIZACIÓN DESORDENADA:
   - LAS INSTRUCCIONES PUEDEN EMITIRSE Y FINALIZARSE EN CUALQUIER ORDEN.
   - VENTAJAS: MAXIMIZA EL PARALELISMO Y LA UTILIZACIÓN DEL HARDWARE.
   - DESVENTAJAS: NECESITA MECANISMOS AVANZADOS COMO RENOMBRAMIENTO DE REGISTROS PARA MANEJAR DEPENDENCIAS Y RIESGOS.

UNA ALTERNATIVA CLAVE ES EL RENOMBRAMIENTO DE REGISTROS, ESENCIAL PARA PROCESADORES CON EMISIÓN Y FINALIZACIÓN DESORDENADA. ESTA TÉCNICA ASIGNA DINÁMICAMENTE REGISTROS FÍSICOS A VALORES TEMPORALES, ELIMINANDO CONFLICTOS COMO DEPENDENCIAS WAR Y WAW. ESTO PERMITE QUE LAS INSTRUCCIONES INDEPENDIENTES SE PROCESEN EN PARALELO, OPTIMIZANDO EL RENDIMIENTO DEL PROCESADOR. AUNQUE AUMENTA LA COMPLEJIDAD DEL HARDWARE, SU IMPACTO EN LA REDUCCIÓN DE RIESGOS LO CONVIERTE EN UNA SOLUCIÓN IDEAL PARA MAXIMIZAR EL PARALELISMO.

¿Qué características posee un multiprocesador simétrico (SMP)?

UN SMP (MULTIPROCESADOR SIMÉTRICO) ES UN SISTEMA CON LAS SIGUIENTES CARACTERÍSTICAS:

- PROCESADORES SIMILARES: TIENE DOS O MÁS PROCESADORES CON CAPACIDADES COMPARABLES.
- MEMORIA COMPARTIDA: LOS PROCESADORES COMPARTEN LA MEMORIA PRINCIPAL Y LOS DISPOSITIVOS DE ENTRADA/SALIDA.
- INTERCONEXIÓN: LOS PROCESADORES ESTÁN CONECTADOS A TRAVÉS DE UN BUS O SISTEMA DE INTERCONEXIÓN.
- TIEMPO DE ACCESO UNIFORME A MEMORIA (UMA): EL TIEMPO DE ACCESO A LA MEMORIA ES SIMILAR PARA TODOS LOS PROCESADORES.
- FUNCIONES EQUIVALENTES: TODOS LOS PROCESADORES PUEDEN REALIZAR LAS MISMAS FUNCIONES.
- SISTEMA OPERATIVO INTEGRADO: PERMITE LA INTERACCIÓN ENTRE LOS PROCESADORES Y LOS PROGRAMAS QUE EJECUTAN.

Describa las 4 variantes de arquitectura de la Taxonomía de Flynn.

SISD (SINGLE INSTRUCTION, SINGLE DATA):

- UN ÚNICO PROCESADOR EJECUTA UNA SOLA SECUENCIA DE INSTRUCCIONES EN UNA ÚNICA SECUENCIA DE DATOS.
- EJEMPLO TÍPICO: COMPUTADORAS MONOPROCESADOR.

SIMD (SINGLE INSTRUCTION, MULTIPLE DATA):

- UNA ÚNICA INSTRUCCIÓN CONTROLA LA EJECUCIÓN SIMULTÁNEA DE MÚLTIPLES PROCESADORES, CADA UNO PROCESANDO SU PROPIA SECUENCIA DE DATOS.
- SE USA EN PROCESADORES VECTORIALES Y MATRICIALES, COMO EN APLICACIONES DE PROCESAMIENTO DE IMÁGENES.

MISD (MULTIPLE INSTRUCTION, SINGLE DATA):

- MÚLTIPLES PROCESADORES EJECUTAN DIFERENTES INSTRUCCIONES SOBRE LA MISMA SECUENCIA DE DATOS.
- ES UNA ESTRUCTURA TEÓRICA QUE NO SE HA IMPLEMENTADO AMPLIAMENTE.

MIMD (MULTIPLE INSTRUCTION, MULTIPLE DATA):

- MÚLTIPLES PROCESADORES EJECUTAN DIFERENTES INSTRUCCIONES EN DISTINTAS SECUENCIAS DE DATOS SIMULTÁNEAMENTE.
- SE DIVIDE EN ARQUITECTURAS CON MEMORIA COMPARTIDA (COMO SMP) Y MEMORIA DISTRIBUIDA (COMO CLÚSTERES).

Si se pretende mejorar el tiempo de acceso medio a memoria cache. ¿Sobre qué parámetros será necesario trabajar y propone como medida para hacerlo?

PARA MEJORAR EL TIEMPO DE ACCESO MEDIO A MEMORIA CACHÉ, ES NECESARIO TRABAJAR SOBRE LOS SIGUIENTES PARÁMETROS, SEGÚN LA FÓRMULA:

$$T_{acceso} = T_{acierto} + TF \times PF$$

1. TIEMPO EN CASO DE ACIERTO (TACIERTO):

- MEDIDA: REDUCIR EL TIEMPO NECESARIO PARA ACCEDER A DATOS EN LA CACHÉ, OPTIMIZANDO EL DISEÑO Y LA ORGANIZACIÓN DE LA MISMA.

2. TASA DE FALLOS (TF):

- MEDIDA: INCREMENTAR EL TAMAÑO DE LA CACHÉ O USAR POLÍTICAS DE CORRESPONDENCIA MÁS FLEXIBLES, COMO LA CORRESPONDENCIA ASOCIATIVA O ASOCIATIVA POR CONJUNTOS, PARA MINIMIZAR LOS CONFLICTOS ENTRE BLOQUES DE DATOS.

3. PENALIZACIÓN POR FALLO (PF):

- MEDIDA: REDUCIR EL TIEMPO NECESARIO PARA ACCEDER A LA MEMORIA PRINCIPAL, MEJORANDO EL ANCHO DE BANDA Y MINIMIZANDO LA LATENCIA EN LA TRANSFERENCIA DE DATOS ENTRE LA MEMORIA PRINCIPAL Y LA CACHÉ. TAMBIÉN IMPLEMENTAR UNA JERARQUÍA DE CACHÉS CON MÚLTIPLES NIVELES (L2, L3) PARA DISMINUIR EL IMPACTO DE LOS FALLOS.

Describa el tratamiento a realizar cuando hay múltiples fuentes de interrupciones

CUANDO EXISTEN MÚLTIPLES FUENTES DE INTERRUPCIONES, EL TRATAMIENTO ADECUADO DEBE CONSIDERAR LAS SIGUIENTES ESTRATEGIAS:

1. INHABILITACIÓN DE INTERRUPCIONES DURANTE EL MANEJO:
   - MIENTRAS SE GESTIONA UNA INTERRUPCIÓN, EL PROCESADOR PUEDE IGNORAR NUEVAS SOLICITUDES DE INTERRUPCIÓN.
   - UNA VEZ FINALIZADA LA GESTIÓN DE LA INTERRUPCIÓN ACTUAL, SE REACTIVAN LAS INTERRUPCIONES PENDIENTES, ASEGURANDO UN MANEJO SECUENCIAL ESTRICTO.
2. DEFINICIÓN DE PRIORIDADES:
   - SE ASIGNAN PRIORIDADES A LAS INTERRUPCIONES. LAS DE MAYOR PRIORIDAD PUEDEN INTERRUMPIR LA GESTIÓN DE LAS DE MENOR PRIORIDAD.
   - EL PROCESADOR MANEJA PRIMERO LAS INTERRUPCIONES MÁS IMPORTANTES Y, AL FINAL, RETOMA LAS DE MENOR PRIORIDAD.
3. MANEJO SECUENCIAL O ANIDADO:
   - SECUENCIAL: LAS INTERRUPCIONES SE PROCESAN EN EL ORDEN EN QUE FUERON RECIBIDAS, ASEGURANDO UN FLUJO LINEAL EN SU MANEJO.
   - ANIDADO: LAS INTERRUPCIONES MÁS PRIORITARIAS PUEDEN INTERRUMPIR LA GESTIÓN DE OTRAS MENOS PRIORITARIAS, PERMITIENDO UN TRATAMIENTO JERÁRQUICO.
4. RECONOCIMIENTO DE INTERRUPCIONES:
   - MULTINIVEL: CADA DISPOSITIVO TIENE UNA LÍNEA FÍSICA CONECTADA A LA CPU, LO QUE PERMITE IDENTIFICAR RÁPIDAMENTE EL ORIGEN DE LA INTERRUPCIÓN, AUNQUE IMPLICA UN COSTO ELEVADO.
   - LÍNEA ÚNICA: UNA SOLA LÍNEA PARA TODAS LAS INTERRUPCIONES; EL PROCESADOR IDENTIFICA EL ORIGEN MEDIANTE UNA TÉCNICA DE CONSULTA O ENCUESTA ("POLLING").
   - VECTORIZADAS: EL DISPOSITIVO QUE SOLICITA LA INTERRUPCIÓN ENVÍA UN IDENTIFICADOR ÚNICO (VECTOR) AL BUS DE DATOS, FACILITANDO LA IDENTIFICACIÓN DIRECTA.
5. USO DE CONTROLADORES DE INTERRUPCIONES:
   - UN CONTROLADOR PROGRAMABLE DE INTERRUPCIONES (PIC) ORGANIZA LAS SOLICITUDES Y FACILITA LA GESTIÓN DE INTERRUPCIONES MÚLTIPLES DE MANERA EFICIENTE, ASIGNANDO VECTORES A CADA TIPO DE INTERRUPCIÓN.

¿Qué características describen un cluster de computadoras?

UN CLÚSTER DE COMPUTADORAS ES UN SISTEMA FORMADO POR VARIAS COMPUTADORAS COMPLETAS, LLAMADAS NODOS, QUE TRABAJAN JUNTAS COMO SI FUERAN UNA SOLA MÁQUINA. CADA NODO TIENE SU PROPIO PROCESADOR, MEMORIA Y SISTEMA OPERATIVO, PERO ESTÁN CONECTADOS A TRAVÉS DE UNA RED QUE LES PERMITE COMUNICARSE Y COORDINARSE. LA PRINCIPAL VENTAJA DE UN CLÚSTER ES SU ESCALABILIDAD, YA QUE PUEDES AGREGAR MÁS NODOS SI NECESITAS MÁS CAPACIDAD DE PROCESAMIENTO. TAMBIÉN OFRECE ALTA DISPONIBILIDAD, PORQUE SI UN NODO FALLA, LOS DEMÁS SIGUEN FUNCIONANDO SIN INTERRUMPIR LAS TAREAS. ESTO LO HACE IDEAL PARA APLICACIONES QUE NECESITAN SER CONFIABLES Y MANEJAR GRANDES CANTIDADES DE DATOS O CÁLCULOS.

ADEMÁS, UN CLÚSTER SUELE SER UNA SOLUCIÓN EFICIENTE EN COSTO, YA QUE PERMITE APROVECHAR MÚLTIPLES COMPUTADORAS TRABAJANDO JUNTAS EN LUGAR DE DEPENDER DE UN SOLO SISTEMA MUY COSTOSO. SE UTILIZA EN ÁREAS COMO SIMULACIONES CIENTÍFICAS, PROCESAMIENTO DE GRANDES BASES DE DATOS Y SERVICIOS EN LA NUBE QUE NECESITAN ESTAR DISPONIBLES TODO EL TIEMPO.

¿Qué características posee un procesador supersegmentado frente a un superescalar?

UN PROCESADOR SUPERSEGMENTADO SE CARACTERIZA POR DIVIDIR CADA ETAPA "MACRO" DEL PIPELINE SEGMENTADO EN SUB-ETAPAS MÁS PEQUEÑAS Y RÁPIDAS, LO QUE PERMITE TRABAJAR CON CICLOS DE RELOJ MÁS CORTOS Y AUMENTAR LA FRECUENCIA DEL RELOJ. ESTO NO REDUCE EL TIEMPO TOTAL PARA COMPLETAR UNA INSTRUCCIÓN, PERO SÍ INCREMENTA EL NIVEL DE PARALELISMO Y LA PERCEPCIÓN DE VELOCIDAD AL MANEJAR MÁS OPERACIONES SIMULTÁNEAMENTE DENTRO DEL PIPELINE.
POR OTRO LADO, UN PROCESADOR SUPERESCALAR BUSCA EJECUTAR VARIAS INSTRUCCIONES AL MISMO TIEMPO, DUPLICANDO O MULTIPLICANDO LAS UNIDADES FUNCIONALES (COMO LAS ALU) PARA PERMITIR QUE DIFERENTES OPERACIONES, COMO SUMAS, MULTIPLICACIONES O ACCESO A MEMORIA, SE PROCESEN EN PARALELO. EL NIVEL DE PARALELISMO EN ESTE CASO DEPENDE DE CUÁNTAS INSTRUCCIONES SE PUEDEN CAPTAR Y EJECUTAR SIMULTÁNEAMENTE.
EN RESUMEN, LA DIFERENCIA PRINCIPAL ES QUE LOS PROCESADORES SUPERSEGMENTADOS MEJORAN EL PARALELISMO DENTRO DE CADA ETAPA DEL PIPELINE AL SUBDIVIDIRLAS, MIENTRAS QUE LOS SUPERESCALARES LO LOGRAN EJECUTANDO MÚLTIPLES INSTRUCCIONES EN PARALELO MEDIANTE RECURSOS DUPLICADOS.

En un cauce segmentado, con secuencia de instrucciones independientes ¿Qué consecuencias trae el paso de una instrucción de salto? Analice los casos de salto incondicional y condicional. Mencione que posibles soluciones se pueden aplicar para evitar o disminuir las consecuencias.

EN UN CAUCE SEGMENTADO, UNA INSTRUCCIÓN DE SALTO PUEDE GENERAR PROBLEMAS PORQUE ALTERA EL FLUJO NORMAL DE LAS INSTRUCCIONES. CUANDO OCURRE UN SALTO, EL VALOR DEL CONTADOR DE PROGRAMA (PC) CAMBIA, LO QUE AFECTA LA CAPTACIÓN DE LAS SIGUIENTES INSTRUCCIONES EN EL CAUCE.
CONSECUENCIAS DEL SALTO:

1. SALTO INCONDICIONAL:
   - SE EJECUTA SIEMPRE, SIN IMPORTAR NINGUNA CONDICIÓN.
   - EN ESTE CASO, EL CAUCE DEBE DESCARTAR TODAS LAS INSTRUCCIONES QUE SE HABÍAN COMENZADO A CARGAR DESPUÉS DEL SALTO. ESTO GENERA BURBUJEO EN EL CAUCE, REDUCIENDO LA EFICIENCIA DEL PROCESADOR.
2. SALTO CONDICIONAL:
   - SOLO SE EJECUTA SI SE CUMPLE UNA CONDICIÓN ESPECÍFICA.
   - AQUÍ, EL PROCESADOR NO PUEDE SABER DE INMEDIATO SI EL SALTO DEBE REALIZARSE O NO, LO QUE GENERA UN RIESGO DE CONTROL. ESTO TAMBIÉN PUEDE LLEVAR A BURBUJEO SI LAS INSTRUCCIONES QUE SE CARGARON RESULTAN NO SER NECESARIAS.

POSIBLES SOLUCIONES:

- PREDICCIÓN DE SALTOS:
  - EL PROCESADOR "ADIVINA" SI EL SALTO OCURRIRÁ O NO Y CONTINÚA CARGANDO INSTRUCCIONES BASÁNDOSE EN ESA PREDICCIÓN. SI LA PREDICCIÓN ES CORRECTA, SE EVITAN RETRASOS; SI NO, SE DESCARTAN LAS INSTRUCCIONES Y SE CORRIGE EL FLUJO.
- EJECUCIÓN ESPECULATIVA:
  - EL PROCESADOR COMIENZA A EJECUTAR AMBAS POSIBLES RUTAS DEL SALTO (LA QUE SIGUE EL SALTO Y LA QUE NO) Y DESPUÉS DESCARTA LA QUE NO SE NECESITA.
- DELAYS SLOTS:
  - SE REORDENA EL CÓDIGO PARA QUE LAS INSTRUCCIONES QUE SIGUEN INMEDIATAMENTE A UN SALTO SEAN ÚTILES Y NO DEPENDAN DEL RESULTADO DEL MISMO.

Describa cómo se debe implementar la estructura de pila en un procesador de tipo RISC cuyos registros son genéricos (basarse en MIPS) ¿Cómo se deberá trabajar el anidamiento de procesos / funciones?

EN UN PROCESADOR RISC COMO MIPS, LA PILA NO ESTÁ IMPLEMENTADA DIRECTAMENTE EN HARDWARE, SINO QUE SE MANEJA MEDIANTE CONVENCIONES DE SOFTWARE USANDO LOS REGISTROS Y LA MEMORIA. DADO QUE LOS PROCESADORES RISC ESTÁN DISEÑADOS PARA TENER UN AMPLIO BANCO DE REGISTROS, SE APROVECHAN LAS "VENTANAS DE REGISTRO" PARA MANEJAR EFICIENTEMENTE LAS VARIABLES LOCALES, LOS PARÁMETROS Y LOS VALORES TEMPORALES DE LAS FUNCIONES.

IMPLEMENTACIÓN DE LA PILA

1. ESTRUCTURA DE REGISTROS Y MEMORIA:
   - LOS REGISTROS SE DIVIDEN EN ÁREAS ESPECÍFICAS PARA PARÁMETROS DE ENTRADA, DATOS LOCALES Y VALORES TEMPORALES.
   - LOS REGISTROS TEMPORALES DE UNA FUNCIÓN PUEDEN SOLAPARSE CON LOS REGISTROS DE PARÁMETROS DE LA FUNCIÓN LLAMADA, LO QUE PERMITE PASAR DATOS SIN NECESIDAD DE ESCRIBIR Y LEER DESDE LA MEMORIA.
2. GESTIÓN DE LA PILA:
   - LA PILA COMO TAL RESIDE EN LA MEMORIA, Y SU MANEJO SE REALIZA MEDIANTE UN REGISTRO ESPECIAL LLAMADO STACK POINTER (SP).
   - AL REALIZAR UNA LLAMADA A FUNCIÓN, LOS VALORES QUE NO CABEN EN LOS REGISTROS O QUE NECESITAN PERSISTIR SE GUARDAN EN LA PILA, MIENTRAS QUE LAS VARIABLES MÁS COMUNES PERMANECEN EN LOS REGISTROS.
3. LLAMADA Y RETORNO DE FUNCIONES:
   - ANTES DE UNA LLAMADA, LOS PARÁMETROS SE COLOCAN EN REGISTROS O, SI SON DEMASIADOS, EN LA PILA.
   - EL CONTEXTO DE LA FUNCIÓN (REGISTROS USADOS) PUEDE GUARDARSE EN LA PILA PARA RESTAURARLO TRAS EL RETORNO.
   - AL RETORNAR, LOS REGISTROS QUE SE USARON PARA LA FUNCIÓN SE RESTAURAN Y EL PUNTERO DE PILA SE AJUSTA.

MANEJO DEL ANIDAMIENTO DE FUNCIONES

EL ANIDAMIENTO SE MANEJA GRACIAS A LAS VENTANAS DE REGISTRO Y LA PILA:

1. VENTANAS DE REGISTRO:
   - CADA FUNCIÓN UTILIZA UN CONJUNTO DE REGISTROS DIFERENTES, Y ESTOS CONJUNTOS SE AJUSTAN AUTOMÁTICAMENTE AL ENTRAR Y SALIR DE FUNCIONES.
   - LOS REGISTROS DE PARÁMETROS DEL NIVEL SUPERIOR SE SOLAPAN CON LOS REGISTROS TEMPORALES DEL NIVEL INFERIOR PARA FACILITAR EL PASO DE PARÁMETROS.
2. PROFUNDIDAD DE LLAMADAS:
   - EL DISEÑO LIMITA EL NÚMERO DE NIVELES DE ANIDAMIENTO QUE SE PUEDEN MANEJAR COMPLETAMENTE EN REGISTROS. PARA NIVELES MÁS PROFUNDOS, EL CONTENIDO SE MUEVE A LA PILA EN MEMORIA.
3. RESTAURACIÓN DE CONTEXTO:
   - CUANDO SE RETORNA DE UNA FUNCIÓN, LOS REGISTROS DEL NIVEL ANTERIOR SE RESTAURAN PARA CONTINUAR LA EJECUCIÓN DEL PROGRAMA LLAMADOR.

EN RESUMEN, LA COMBINACIÓN DE UN AMPLIO BANCO DE REGISTROS, EL USO DE VENTANAS DE REGISTRO Y EL MANEJO DE LA PILA EN MEMORIA PERMITE QUE LOS PROCESADORES RISC COMO MIPS TRABAJEN EFICIENTEMENTE CON FUNCIONES Y PROCESOS ANIDADOS. ESTE DISEÑO MINIMIZA EL ACCESO A LA MEMORIA Y MAXIMIZA EL USO DE REGISTROS PARA UN RENDIMIENTO ÓPTIMO.

Que diferencias existen ente la E/S mapeada en memoria y la E/S aislada en una computadora?

LA PRINCIPAL DIFERENCIA ENTRE LA E/S MAPEADA EN MEMORIA Y LA E/S AISLADA RADICA EN CÓMO SE GESTIONAN LOS ESPACIOS DE DIRECCIONES Y LAS INSTRUCCIONES PARA INTERACTUAR CON LOS DISPOSITIVOS:

1. E/S MAPEADA EN MEMORIA:
   - LOS DISPOSITIVOS DE E/S COMPARTEN EL MISMO ESPACIO DE DIRECCIONES QUE LA MEMORIA PRINCIPAL.
   - ESTO SIGNIFICA QUE LAS OPERACIONES DE E/S SE PUEDEN REALIZAR UTILIZANDO LAS MISMAS INSTRUCCIONES QUE SE USAN PARA LEER O ESCRIBIR EN MEMORIA.
   - ES MÁS EFICIENTE EN TÉRMINOS DE PROGRAMACIÓN PORQUE NO REQUIERE INSTRUCCIONES ESPECÍFICAS PARA LA E/S.
2. E/S AISLADA:
   - LOS DISPOSITIVOS DE E/S TIENEN UN ESPACIO DE DIRECCIONES SEPARADO DE LA MEMORIA.
   - REQUIERE LÍNEAS Y COMANDOS ESPECÍFICOS DE CONTROL PARA LAS OPERACIONES DE E/S, CON UN CONJUNTO LIMITADO DE INSTRUCCIONES.
   - ESTO PUEDE HACER QUE SEA MENOS FLEXIBLE, PERO PUEDE SIMPLIFICAR EL DISEÑO DEL HARDWARE EN ALGUNOS CASOS.

EN RESUMEN, LA E/S MAPEADA EN MEMORIA INTEGRA LA MEMORIA Y LOS DISPOSITIVOS DE E/S EN UN ÚNICO ESPACIO DE DIRECCIONES, MIENTRAS QUE LA E/S AISLADA MANTIENE ESTOS ESPACIOS SEPARADOS Y UTILIZA INSTRUCCIONES ESPECÍFICAS PARA ACCEDER A LOS DISPOSITIVOS

Justifique el uso de dos niveles de caché

EL USO DE DOS NIVELES DE CACHÉ SE JUSTIFICA PARA MEJORAR EL RENDIMIENTO DEL SISTEMA AL EQUILIBRAR LA VELOCIDAD Y EL COSTO. SEGÚN EL DOCUMENTO, LA JERARQUÍA DE MEMORIA BUSCA QUE LA VELOCIDAD DEL SISTEMA SEA CERCANA A LA DEL NIVEL MÁS RÁPIDO (COMO LA CACHÉ DE NIVEL 1, L1) PERO AL COSTO DE LOS NIVELES MÁS ECONÓMICOS (COMO LA MEMORIA PRINCIPAL).

LA CACHÉ L1 ES MÁS PEQUEÑA Y RÁPIDA, LO QUE PERMITE SERVIR DATOS MÁS UTILIZADOS CON MENOR LATENCIA. LA CACHÉ L2, AUNQUE MÁS LENTA Y GRANDE, ACTÚA COMO RESPALDO PARA LA L1, REDUCIENDO LA FRECUENCIA DE ACCESOS A LA MEMORIA PRINCIPAL, QUE ES MUCHO MÁS LENTA. ESTO APROVECHA EL PRINCIPIO DE LOCALIDAD (TEMPORAL Y ESPACIAL), ALMACENANDO DATOS QUE PROBABLEMENTE SE REUTILICEN EN LOS NIVELES SUPERIORES. ASÍ, SE OPTIMIZAN TANTO LA TASA DE ACIERTOS COMO LA PENALIZACIÓN POR FALLOS.

EN RESUMEN, ESTA ORGANIZACIÓN MEJORA EL TIEMPO PROMEDIO DE ACCESO A MEMORIA, HACIÉNDOLO MÁS EFICIENTE.

Describa las características que diferencian los procesadores RISC de los CISC

1. REPERTORIO DE INSTRUCCIONES:
   - RISC: UTILIZAN UN CONJUNTO REDUCIDO Y SENCILLO DE INSTRUCCIONES, LO QUE FACILITA LA OPTIMIZACIÓN DE LA EJECUCIÓN Y PERMITE QUE CADA INSTRUCCIÓN SE EJECUTE EN UN CICLO DE RELOJ.
   - CISC: INCLUYEN UN REPERTORIO AMPLIO Y COMPLEJO DE INSTRUCCIONES, ALGUNAS DE LAS CUALES EJECUTAN TAREAS COMPLEJAS EN UNA SOLA OPERACIÓN.
2. OPTIMIZACIÓN DEL RENDIMIENTO:
   - RISC: DA PRIORIDAD A LA EFICIENCIA DEL HARDWARE Y A LA SEGMENTACIÓN DE INSTRUCCIONES. ESTO SE LOGRA MEDIANTE UN DISEÑO CABLEADO Y FORMATOS DE INSTRUCCIONES FIJOS Y SENCILLOS.
   - CISC: BUSCA REDUCIR LA CANTIDAD DE INSTRUCCIONES EN EL CÓDIGO FUENTE, FACILITANDO LA PROGRAMACIÓN, PERO A COSTA DE UNA MAYOR COMPLEJIDAD DEL HARDWARE Y UN MICROCÓDIGO MÁS GRANDE.
3. USO DE REGISTROS:
   - RISC: HACE UN USO INTENSIVO DE REGISTROS, LO QUE MINIMIZA EL ACCESO A LA MEMORIA Y MEJORA EL RENDIMIENTO GENERAL.
   - CISC: DEPENDE MÁS DE ACCESOS A MEMORIA DEBIDO A LA NATURALEZA COMPLEJA DE LAS INSTRUCCIONES.
4. TIEMPO DE EJECUCIÓN:
   - RISC: DISEÑADO PARA QUE CADA INSTRUCCIÓN SE EJECUTE EN UN CICLO DE RELOJ, FAVORECIENDO LA VELOCIDAD.
   - CISC: PUEDE TARDAR VARIOS CICLOS DE RELOJ PARA COMPLETAR INSTRUCCIONES COMPLEJAS.
5. MODOS DE DIRECCIONAMIENTO:
   - RISC: TIENE MODOS DE DIRECCIONAMIENTO SIMPLES Y LIMITADOS.
   - CISC: OFRECE UNA MAYOR VARIEDAD DE MODOS DE DIRECCIONAMIENTO, LO QUE INCREMENTA LA FLEXIBILIDAD PERO TAMBIÉN LA COMPLEJIDAD.

EN RESUMEN, RISC BUSCA SIMPLIFICAR LAS INSTRUCCIONES PARA MAXIMIZAR EL RENDIMIENTO A NIVEL DE HARDWARE, MIENTRAS QUE CISC OPTA POR INSTRUCCIONES MÁS COMPLEJAS QUE SIMPLIFIQUEN LA PROGRAMACIÓN Y REDUZCAN EL TAMAÑO DEL CÓDIGO, AUNQUE ESTO IMPLIQUE MAYOR COMPLEJIDAD DEL HARDWARE Y POSIBLES PENALIZACIONES DE RENDIMIENTO EN CIERTOS CASOS.

¿Qué es una pila?

UNA PILA ES UNA ESTRUCTURA DE DATOS QUE OPERA BAJO EL PRINCIPIO DE "ÚLTIMO EN ENTRAR, PRIMERO EN SALIR" (LIFO). ESTO SIGNIFICA QUE EL ÚLTIMO ELEMENTO QUE SE AGREGA A LA PILA ES EL PRIMERO EN SER RETIRADO. PARA MANEJAR UNA PILA, SE UTILIZA UN REGISTRO ESPECIAL LLAMADO PUNTERO DE PILA (SP), QUE SEÑALA LA DIRECCIÓN DEL ÚLTIMO ELEMENTO AGREGADO.

LAS PRINCIPALES OPERACIONES QUE SE REALIZAN EN UNA PILA SON:

- PUSH: AÑADIR UN ELEMENTO A LA CIMA DE LA PILA.
- POP: RETIRAR EL ELEMENTO QUE ESTÁ EN LA CIMA DE LA PILA.

AMBAS OPERACIONES IMPLICAN UN MOVIMIENTO DE DATOS ENTRE REGISTROS Y MEMORIA, Y UNA ACTUALIZACIÓN DEL PUNTERO DE PILA, LO QUE DEFINE CÓMO CRECE O DECRECE LA PILA.

Mencione las principales diferencias entre PCI y SCSI

1. PROPÓSITO PRINCIPAL:

- PCI: ES UN BUS INTERNO DISEÑADO PARA CONECTAR COMPONENTES COMO TARJETAS GRÁFICAS, TARJETAS DE RED Y ALMACENAMIENTO DENTRO DE LA PLACA BASE DE UN SISTEMA.
- SCSI: ES UNA INTERFAZ ESTÁNDAR PARA CONECTAR DISPOSITIVOS PERIFÉRICOS EXTERNOS, COMO DISCOS DUROS, ESCÁNERES E IMPRESORAS.

2. ARQUITECTURA:

- PCI: SE ENFOCA EN CONECTAR DISPOSITIVOS DIRECTAMENTE AL BUS DE LA CPU. ES UNA ARQUITECTURA DE BUS PARALELO.
- SCSI: TIENE SU PROPIO CONTROLADOR QUE PERMITE LA CONEXIÓN DE MÚLTIPLES DISPOSITIVOS EN CADENA (DAISY CHAIN) Y COMUNICACIÓN INDEPENDIENTE DEL CPU.

3. COMPATIBILIDAD Y CONFIGURACIÓN:

- PCI: SE USA PRINCIPALMENTE DENTRO DE COMPUTADORAS PERSONALES Y SERVIDORES MODERNOS.
- SCSI: ES MÁS COMÚN EN ENTORNOS EMPRESARIALES O SERVIDORES ANTIGUOS QUE NECESITAN DISPOSITIVOS EXTERNOS DE ALMACENAMIENTO O ESPECIALIZADOS.

4. VELOCIDAD Y RENDIMIENTO:

- PCI: OFRECE MAYORES VELOCIDADES DE TRANSFERENCIA Y BAJA LATENCIA EN COMPARACIÓN CON SCSI DEBIDO A SU DISEÑO INTERNO Y SU PROXIMIDAD A LA CPU.
- SCSI: AUNQUE MÁS LENTO QUE PCI EN SUS VERSIONES CLÁSICAS, PERMITE LA OPERACIÓN SIMULTÁNEA DE MÚLTIPLES DISPOSITIVOS, LO QUE ES IDEAL PARA SISTEMAS CON ALTA DEMANDA DE PERIFÉRICOS.

Que sucede cuando tenemos varias subrutinas anidadas?

CUANDO TENEMOS VARIAS SUBRUTINAS ANIDADAS, CADA UNA DE ELLAS GENERA UN NUEVO NIVEL EN LA PILA (STACK) PARA MANEJAR SUS PARÁMETROS, DIRECCIONES DE RETORNO Y VARIABLES LOCALES. ESTO IMPLICA LO SIGUIENTE:

1. GESTIÓN DE LA PILA: CADA SUBRUTINA APILA SU INFORMACIÓN EN LA PILA, INCLUYENDO PARÁMETROS, LA DIRECCIÓN DE RETORNO AL PROGRAMA LLAMADOR Y REGISTROS QUE NECESITAN SER PRESERVADOS.
2. CAMBIOS EN LOS PUNTEROS: EL PUNTERO DE PILA (SP) Y, EN ALGUNOS CASOS, EL PUNTERO BASE (BP), SE ACTUALIZAN PARA REFLEJAR EL NUEVO NIVEL DE ANIDACIÓN. ESTO PERMITE QUE CADA SUBRUTINA TENGA SU PROPIO MARCO DE PILA INDEPENDIENTE.
3. REGRESO ORDENADO: CUANDO UNA SUBRUTINA TERMINA, SU MARCO DE PILA SE DESAPILA Y SE RESTAURA EL ESTADO PREVIO (INCLUYENDO REGISTROS Y PUNTEROS), PERMITIENDO QUE EL FLUJO DEL PROGRAMA REGRESE A LA SUBRUTINA O PROGRAMA QUE LA LLAMÓ.
4. COMPLEJIDAD ACUMULADA: A MEDIDA QUE MÁS SUBRUTINAS SE ANIDAN, AUMENTA LA CANTIDAD DE INFORMACIÓN EN LA PILA, LO QUE REQUIERE UN MANEJO CUIDADOSO PARA EVITAR SOBREPASAR LOS LÍMITES DE LA MISMA O CORROMPER DATOS.

ESTE MECANISMO DE ANIDAMIENTO ES FUNDAMENTAL PARA MANTENER LA MODULARIDAD Y EL ORDEN EN PROGRAMAS COMPLEJOS